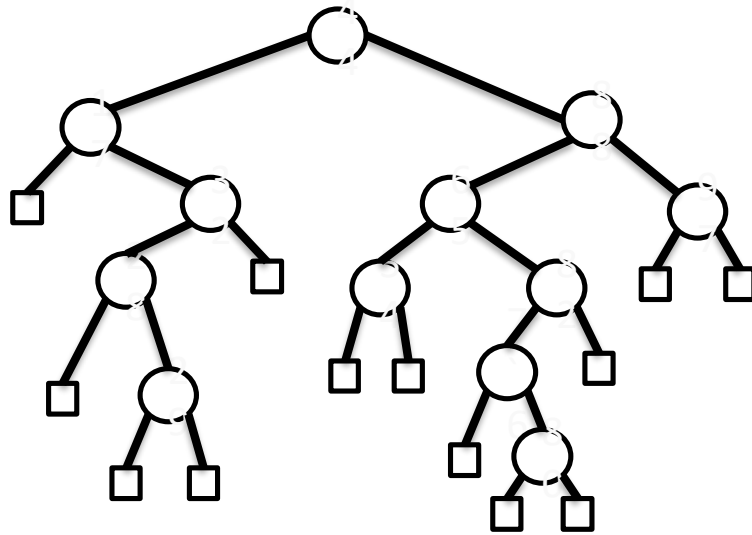
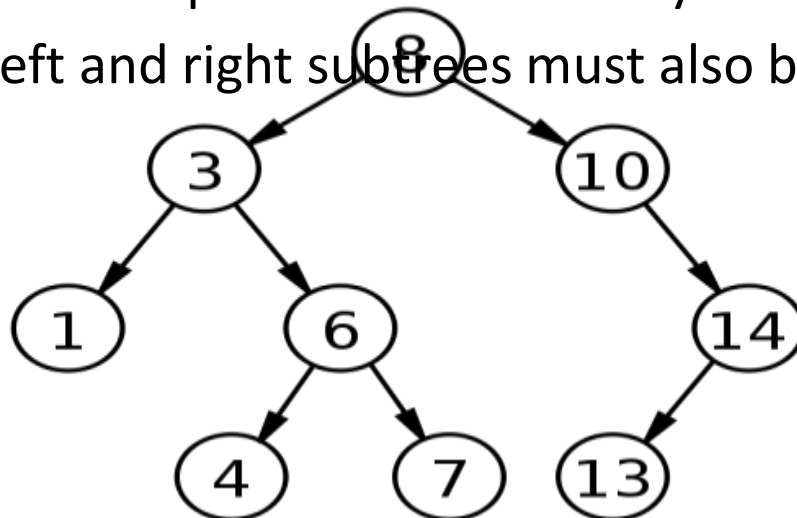


Binary Search Trees



Binary Search Trees

- A **binary search tree (BST)** is a node-based binary tree data structure which has the following properties:
 - The left subtree of a node contains only nodes with keys less than the node's key.
 - The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
 - Both the left and right subtrees must also be binary search trees.

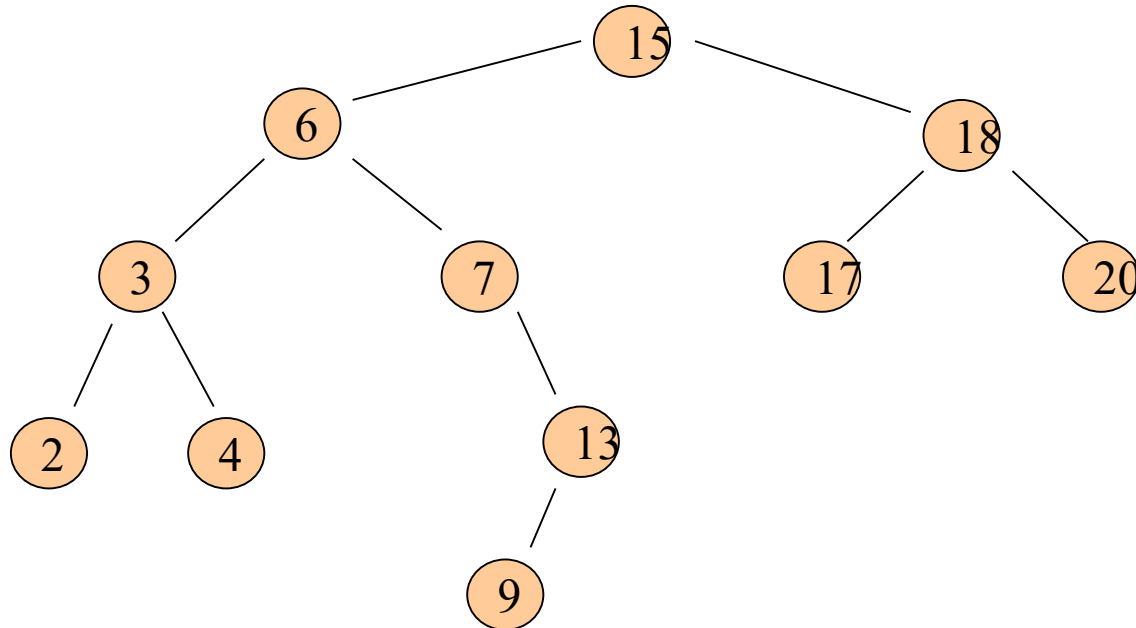


Binary Search Trees

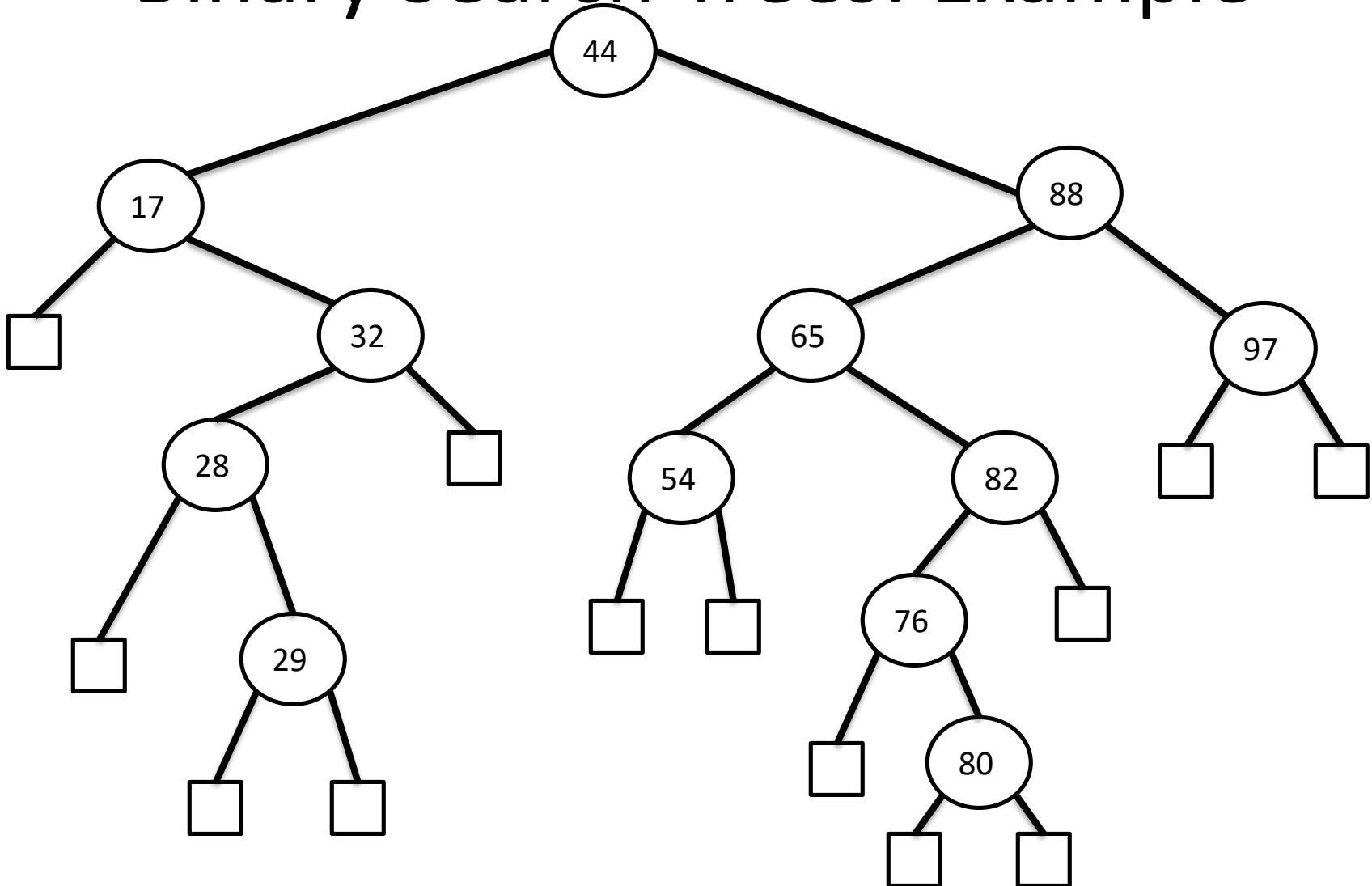
- Firstly, it is a binary tree
- It is represented by a linked data structure
- It combines
 - the advantage of an array -- the ability to do a binary search with
 - the advantage of a linked list -- its dynamic size
- The efficiency of all of the operations is $O(\log n)$, only if the tree is reasonably height-balanced
- Each node contains at least the following fields
 - *key*: an identifying field
 - *left*: pointer to a left child (may be NIL)
 - *right*: pointer to a right child (may be NIL)
 - *p*: pointer to the parent node (NIL for root)

Binary Search Trees

- Secondly, the binary-search-tree property
 - For any node x ,
 - Key $[y] < \text{Key}[x]$ if y in left subtree of x
 - Key $[y] \geq \text{Key}[x]$ if y in right subtree of x

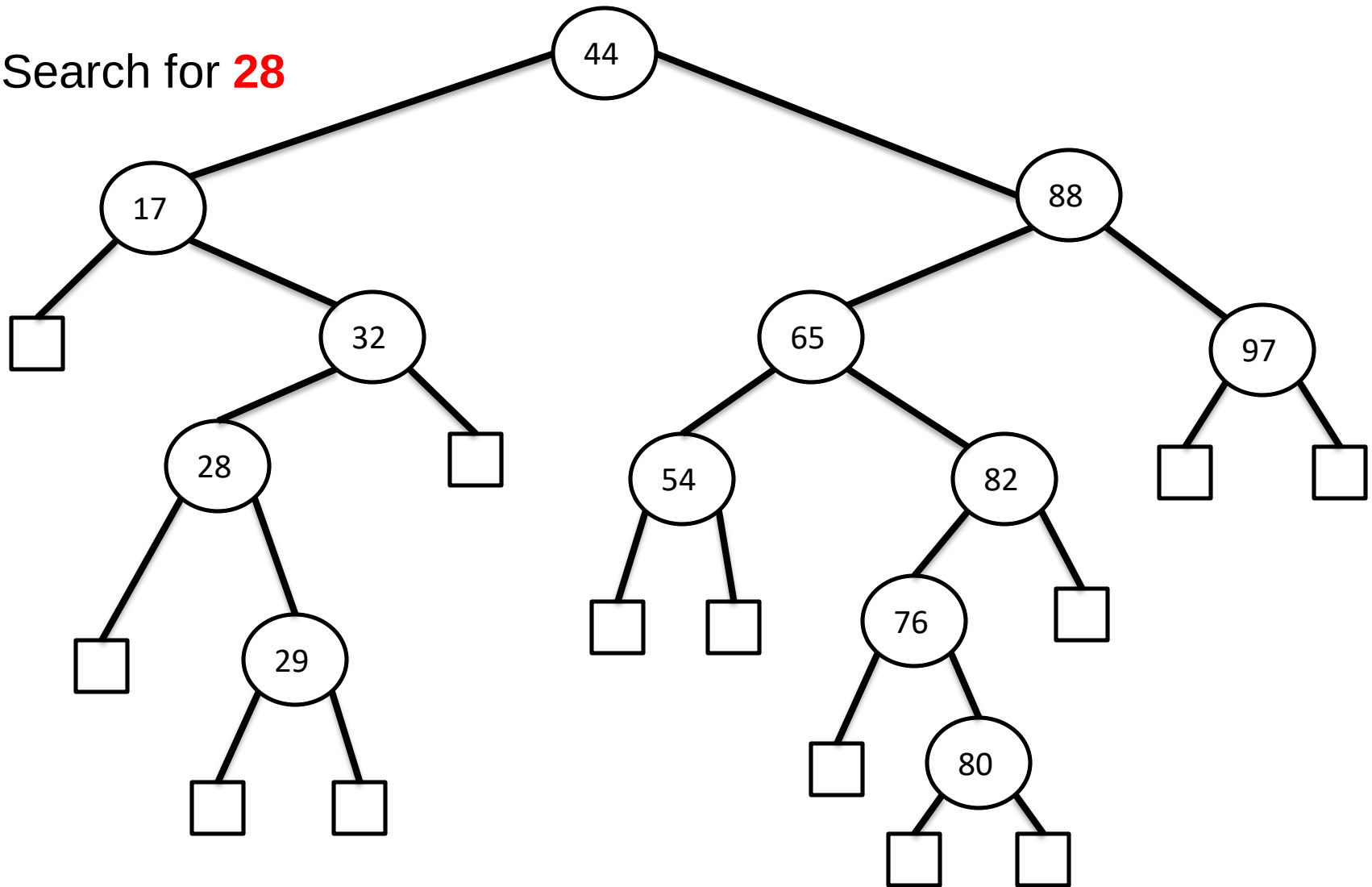


Binary Search Trees: Example

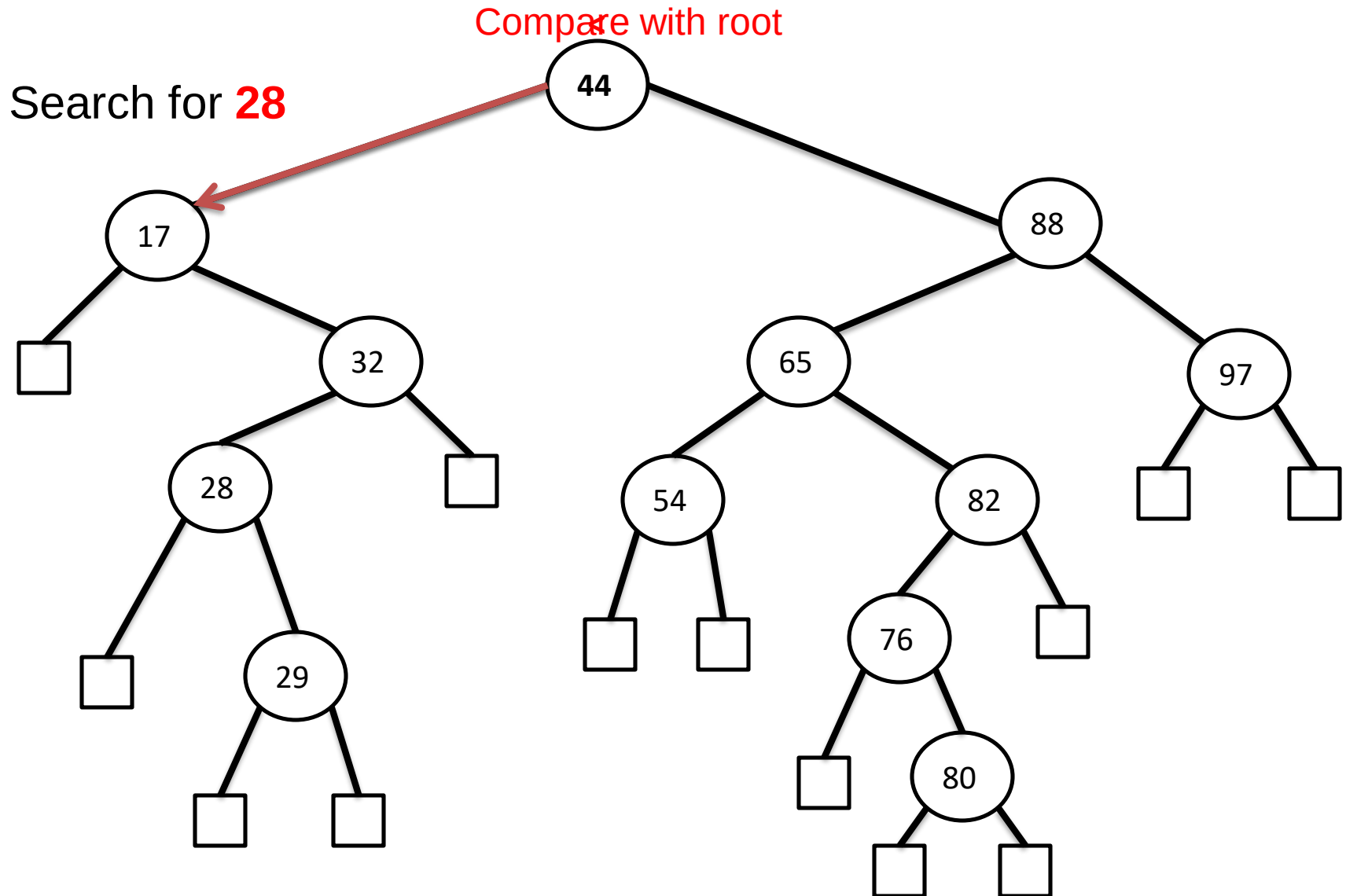


Operations on BSTs: Search

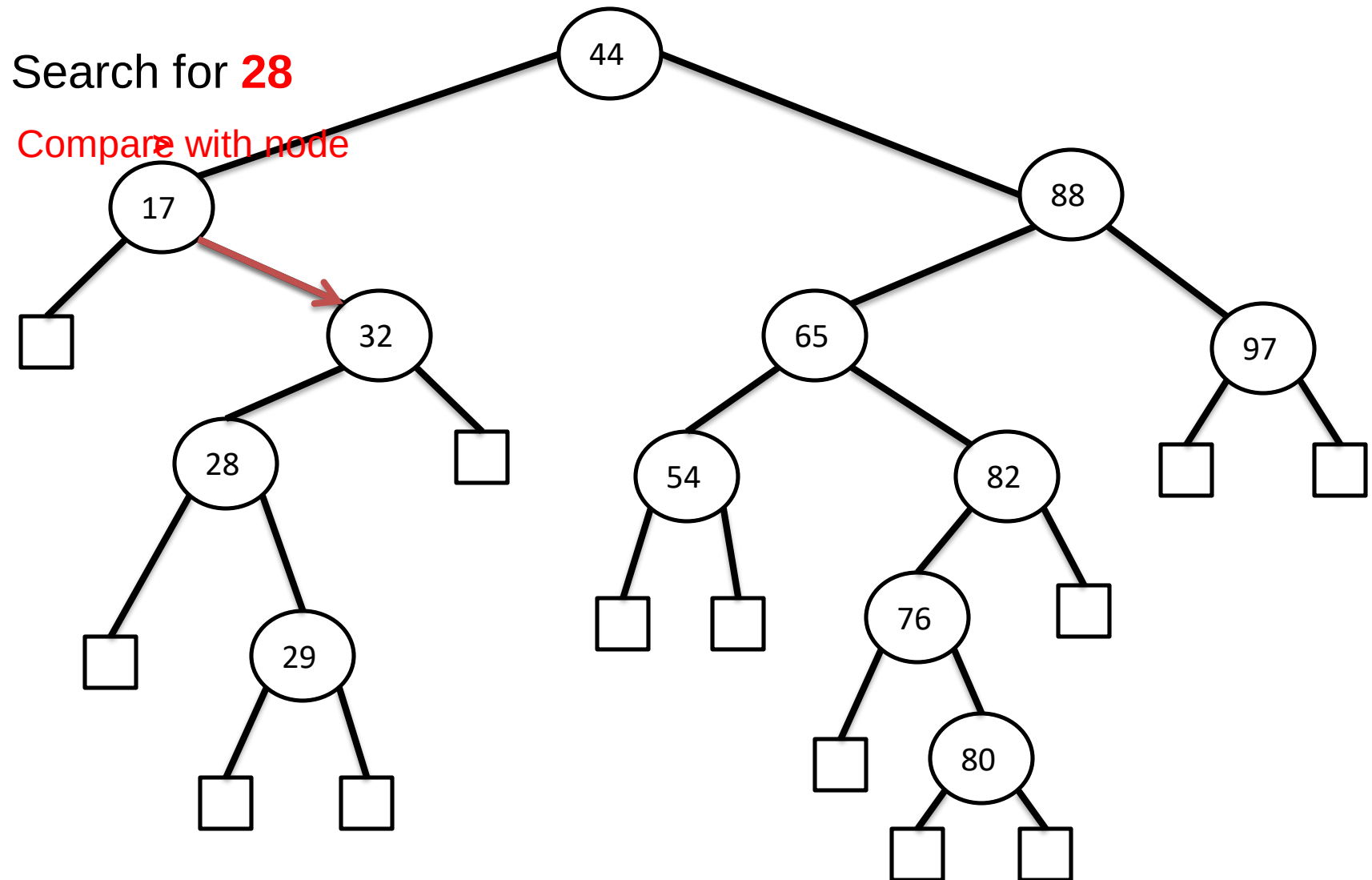
Search for **28**



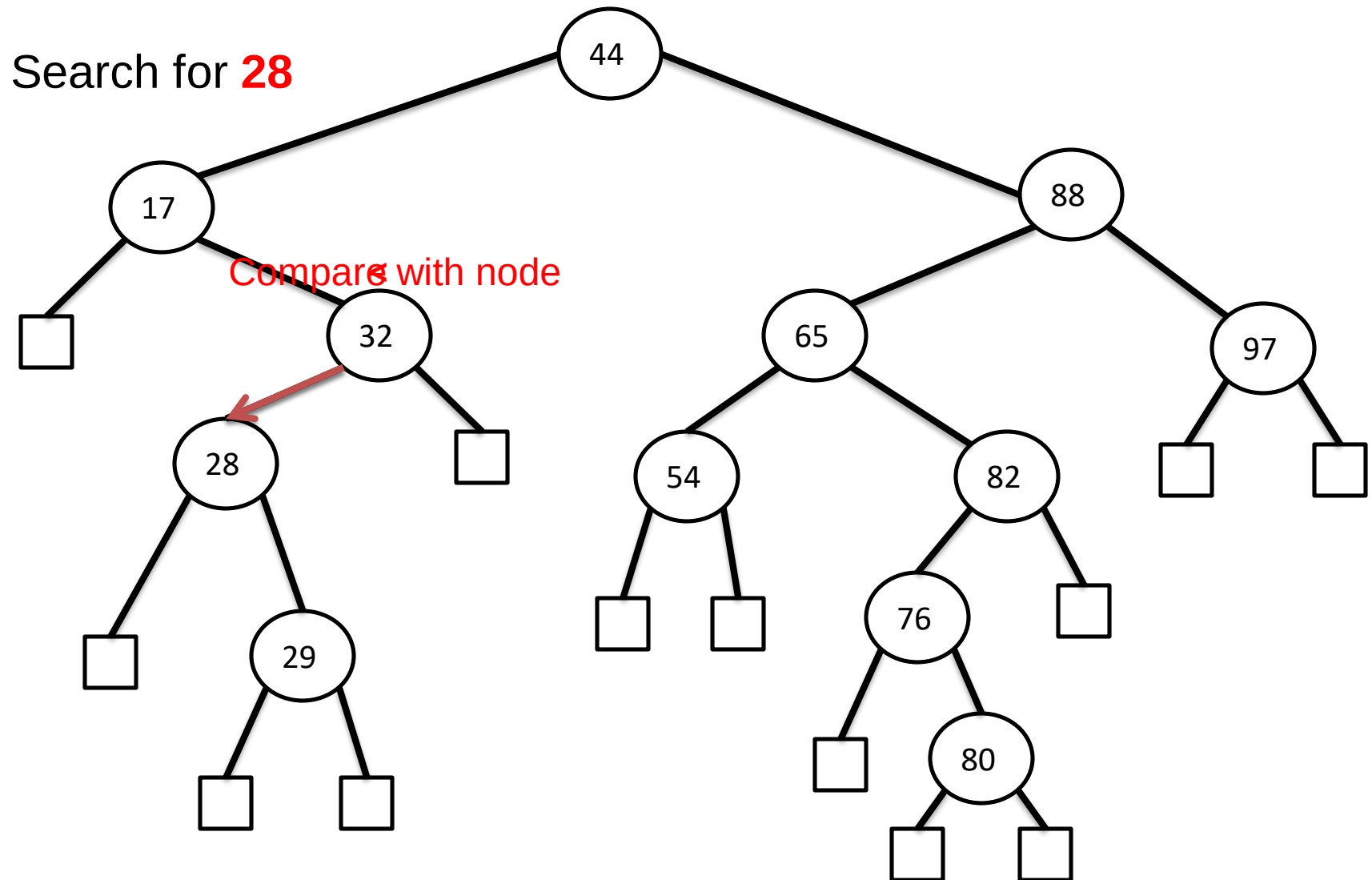
Operations on BSTs: Search



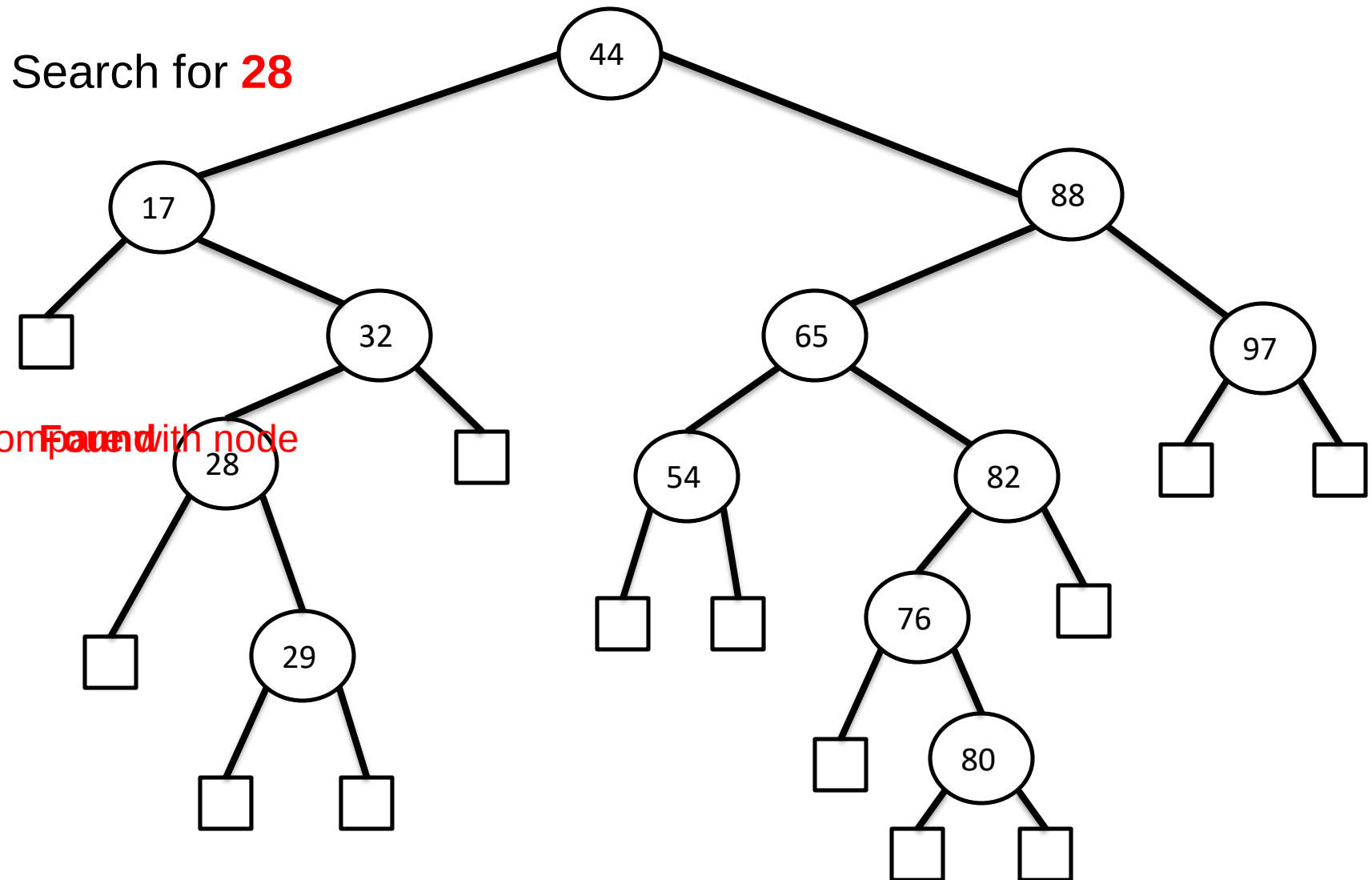
Operations on BSTs: Search



Operations on BSTs: Search



Operations on BSTs: Search



Operations on BSTs: Search

- Given a key k and a pointer to the root, Tree-Search returns a pointer to a node with key k or NIL:

```
Tree-Search(x, k)
    if (x == NIL or k == key[x])
        return x;
    if (k < key[x])
        return Tree-Search(left[x], k);
    else
        return Tree-Search(right[x], k);
```

Operations on BSTs: Search

- Here is another function that does the same:

```
Tree-Search(x, k)
  while (x != NIL and k != key[x])
    if (k < key[x])
      x = left[x];
    else
      x = right[x];
  return x;
```

- *Which of these two functions is more efficient?*

Operations on BSTs: Min & Max

TREE-MINIMUM(x)

```
1  while  $left[x] \neq \text{NIL}$   
2      do  $x \leftarrow left[x]$   
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $right[x] \neq \text{NIL}$   
2      do  $x \leftarrow right[x]$   
3  return  $x$ 
```

Operations on BSTs: Successor

- Two cases:
 - x has a right subtree: successor is minimum node in right subtree
 - x has no right subtree: successor is first ancestor of x whose left child is also ancestor of x
 - Intuition: As long as you move to the left up the tree, you're visiting smaller nodes.
- Predecessor: similar algorithm

Operations on BSTs: Successor

TREE-SUCCESSOR(x)

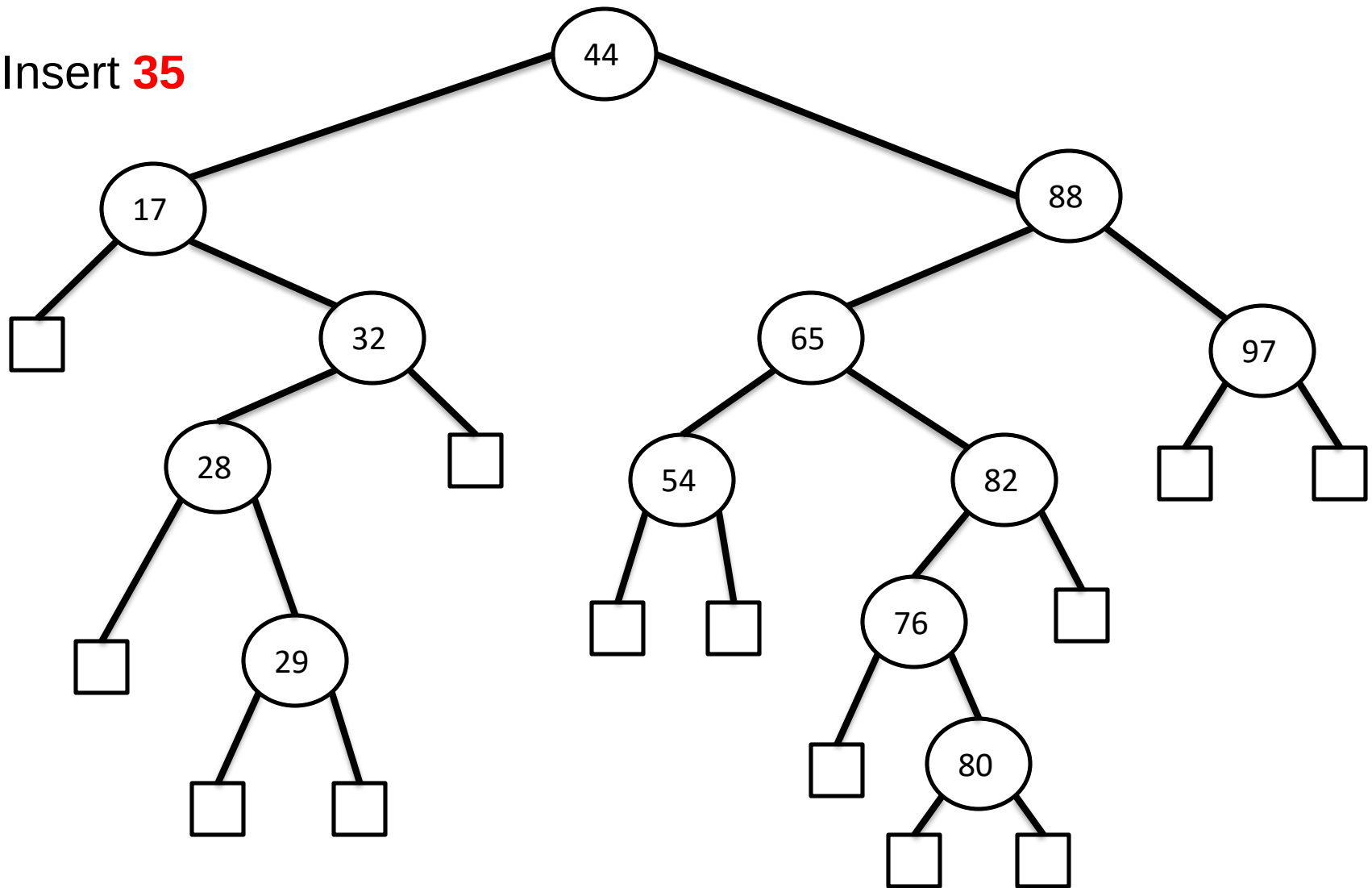
```
1  if  $right[x] \neq \text{NIL}$ 
2      then return TREE-MINIMUM( $right[x]$ )
3   $y \leftarrow p[x]$ 
4  while  $y \neq \text{NIL}$  and  $x = right[y]$ 
5      do  $x \leftarrow y$ 
6       $y \leftarrow p[y]$ 
7  return  $y$ 
```

Operations of BSTs: Insert

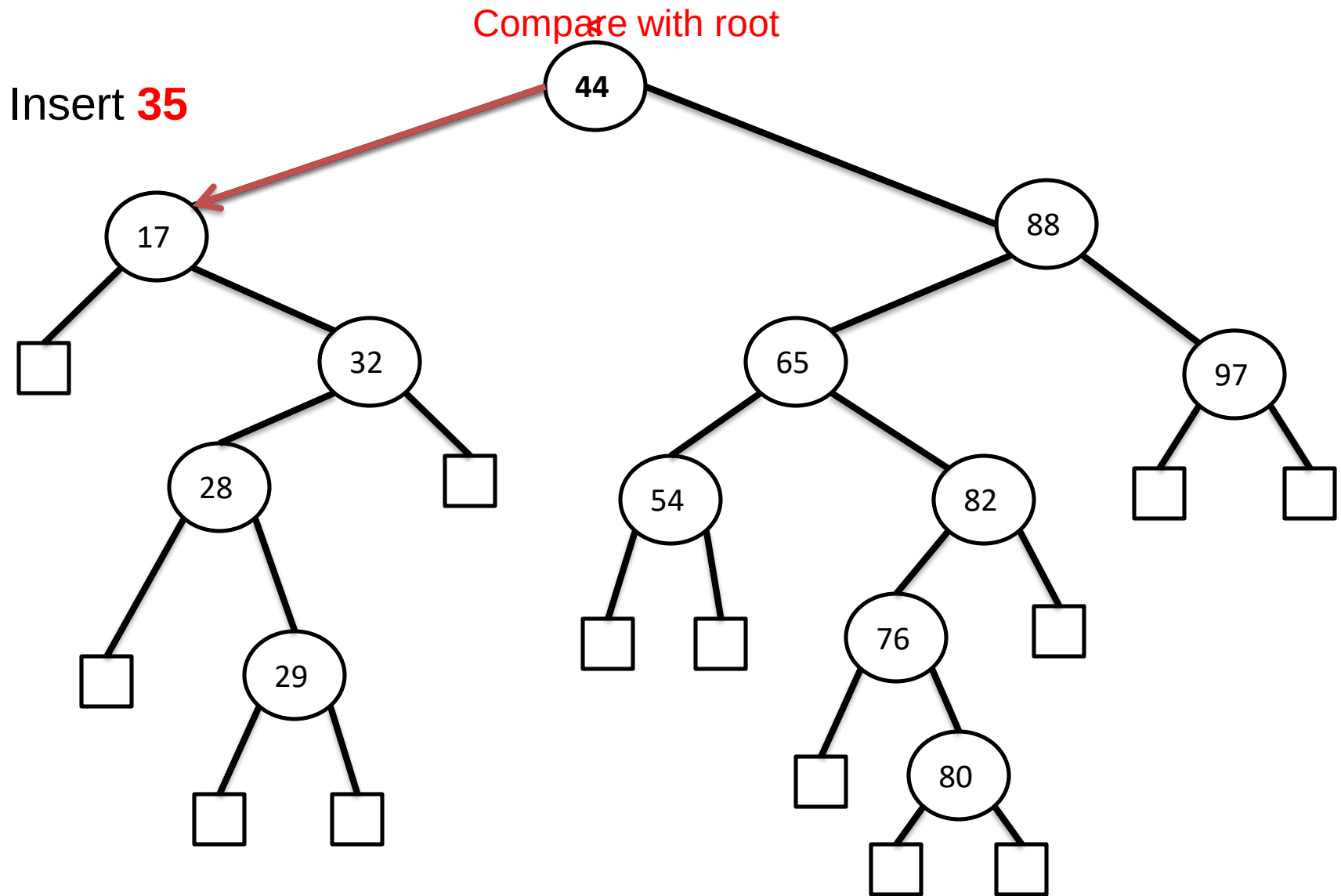
- Inserts an element z to the tree so that the binary search tree property continues to hold
- The basic algorithm
 - Like the search procedure as discussed before
 - Insert z in place of NIL
 - Use a “trailing pointer” to keep track of where you came from (like inserting into singly linked list)

Operations of BSTs: Insert

Insert **35**



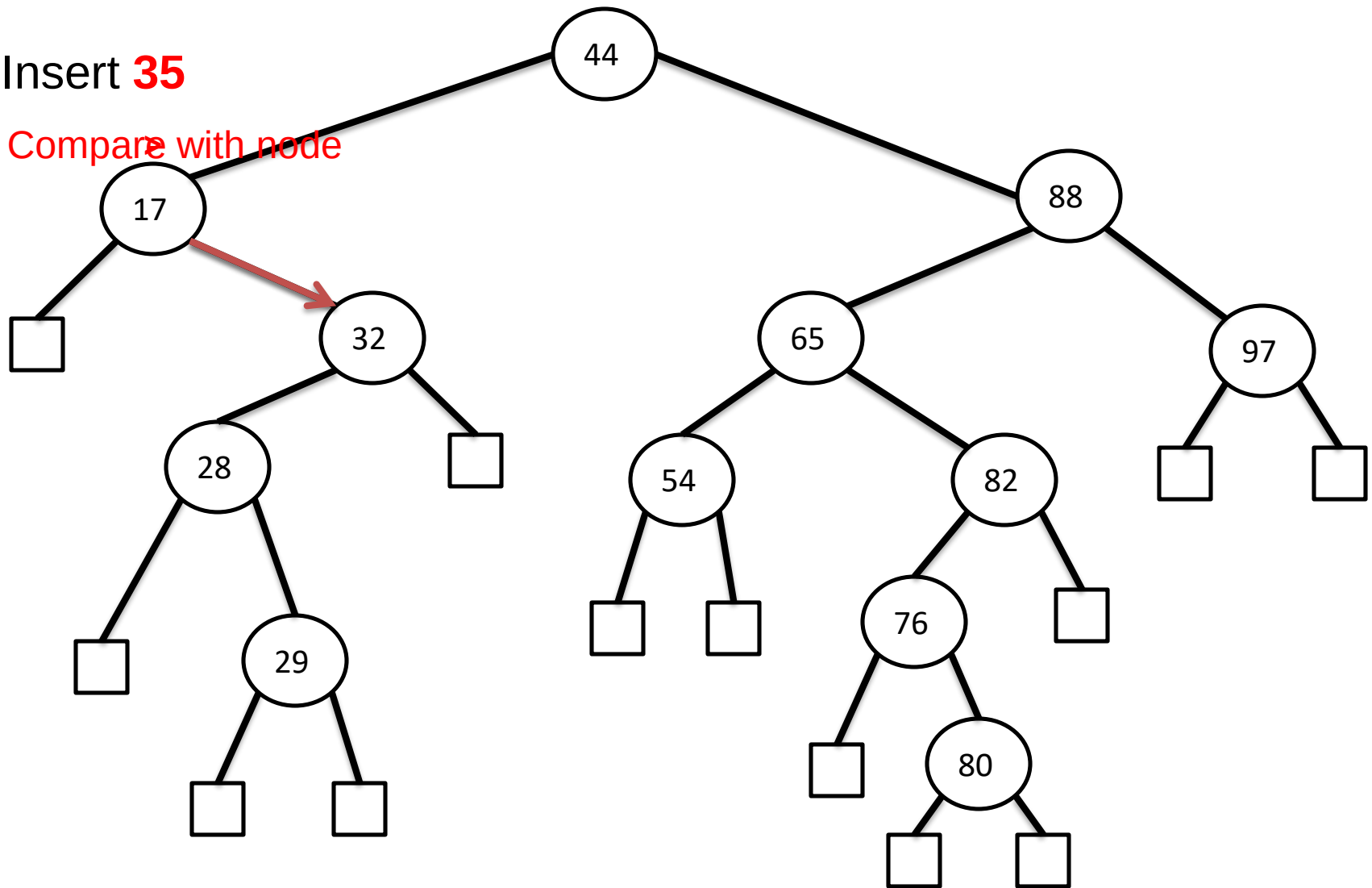
Operations of BSTs: Insert



Operations of BSTs: Insert

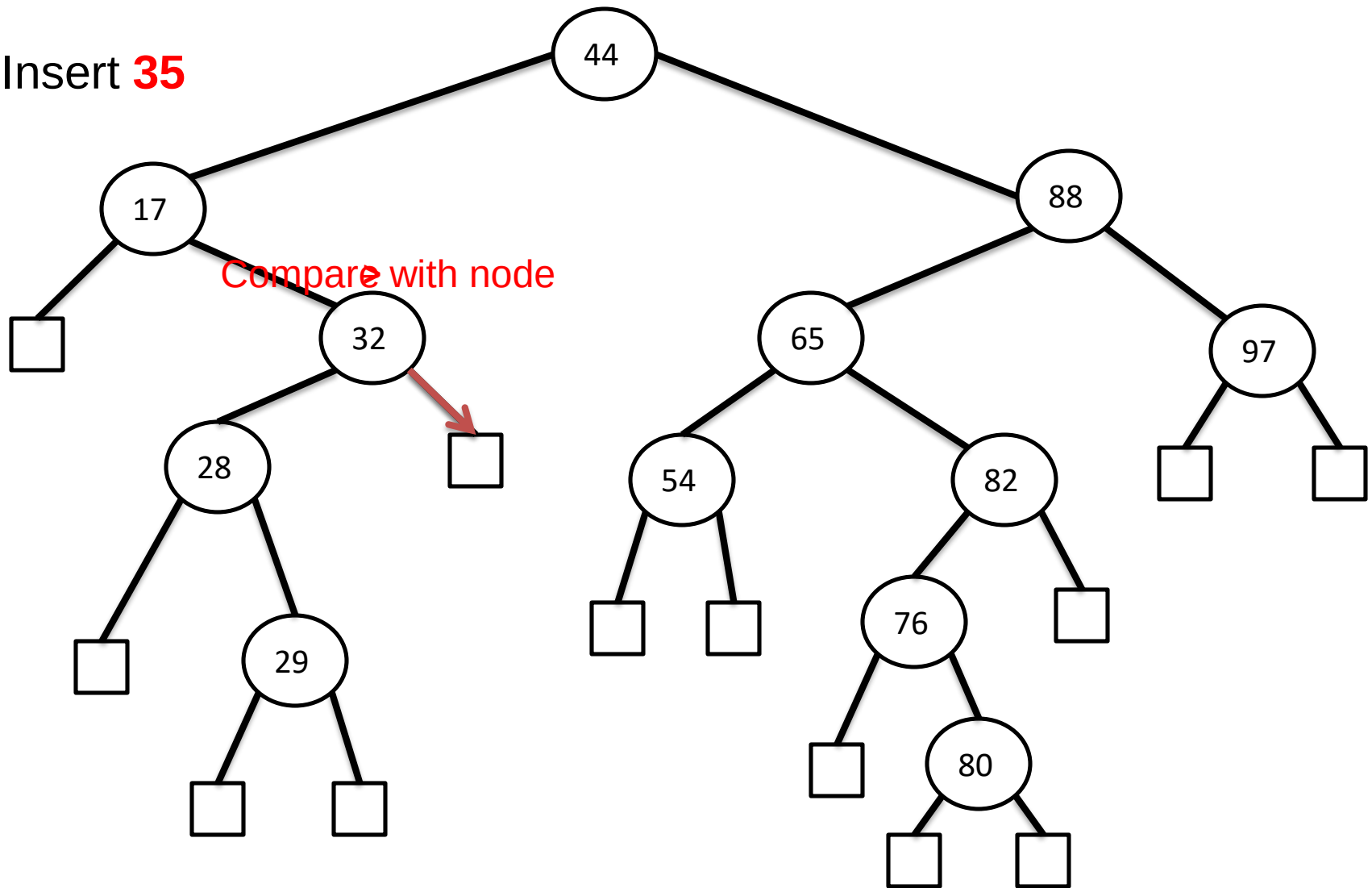
Insert **35**

Compare with node



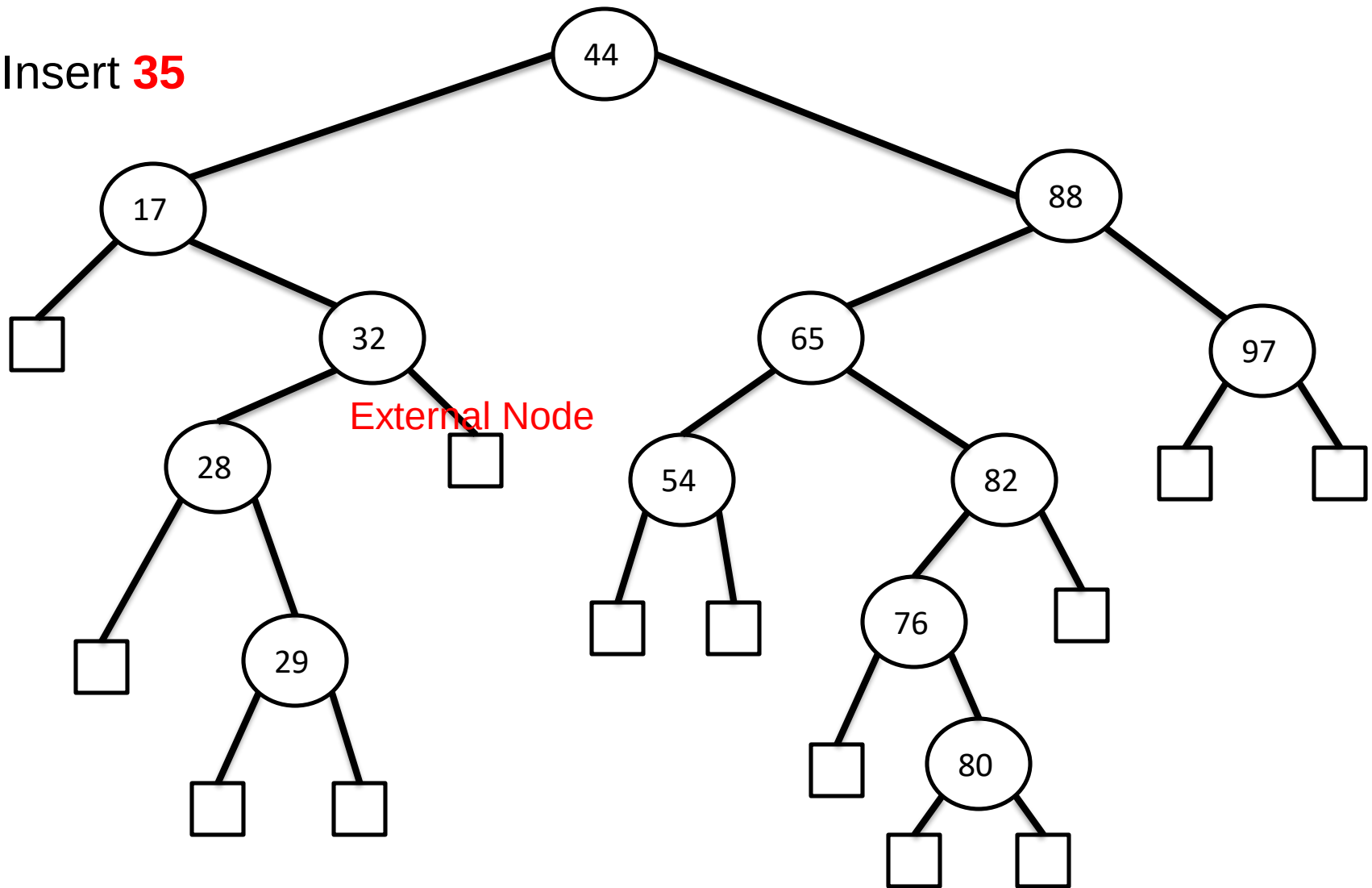
Operations of BSTs: Insert

Insert **35**



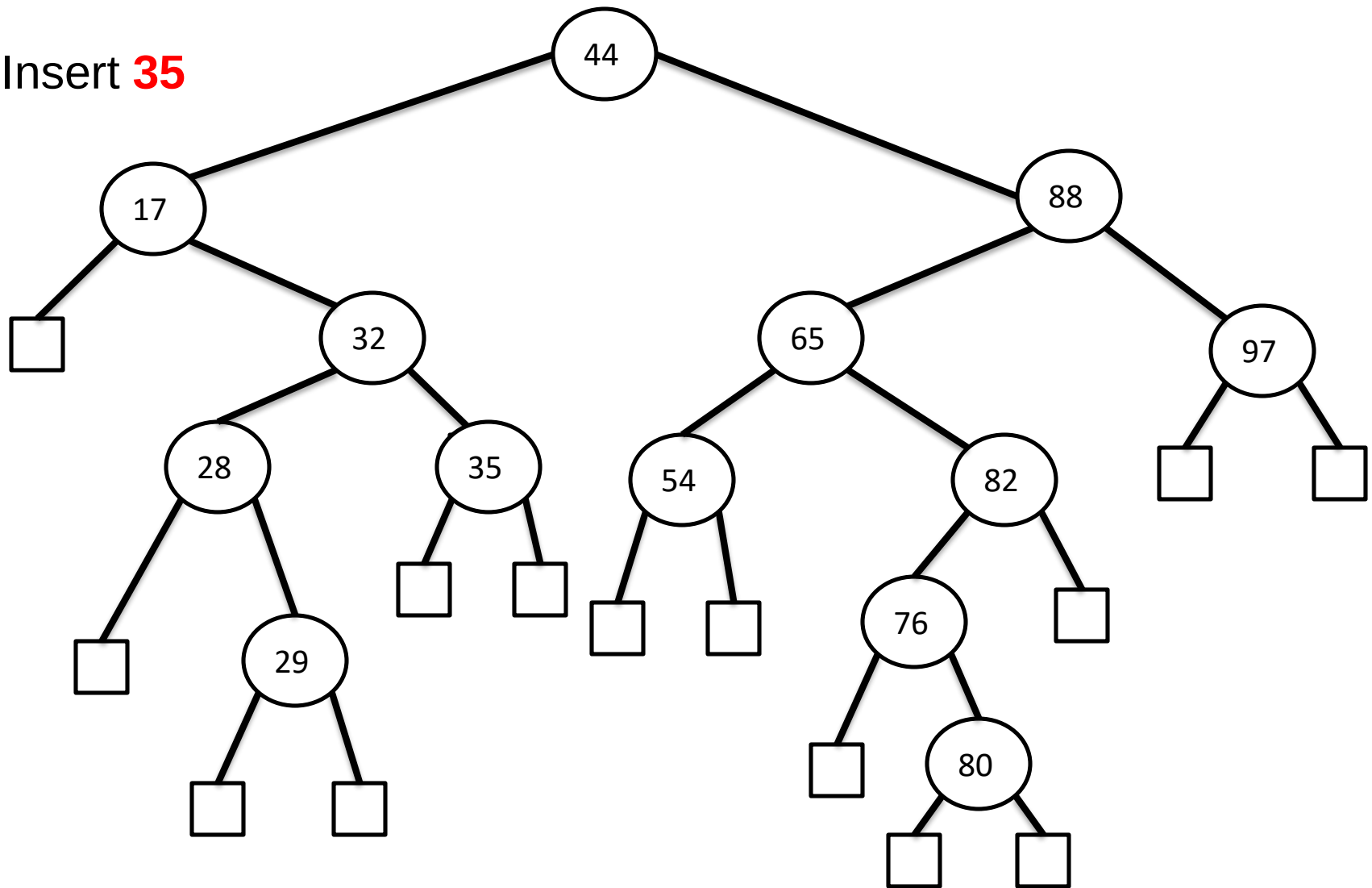
Operations of BSTs: Insert

Insert **35**



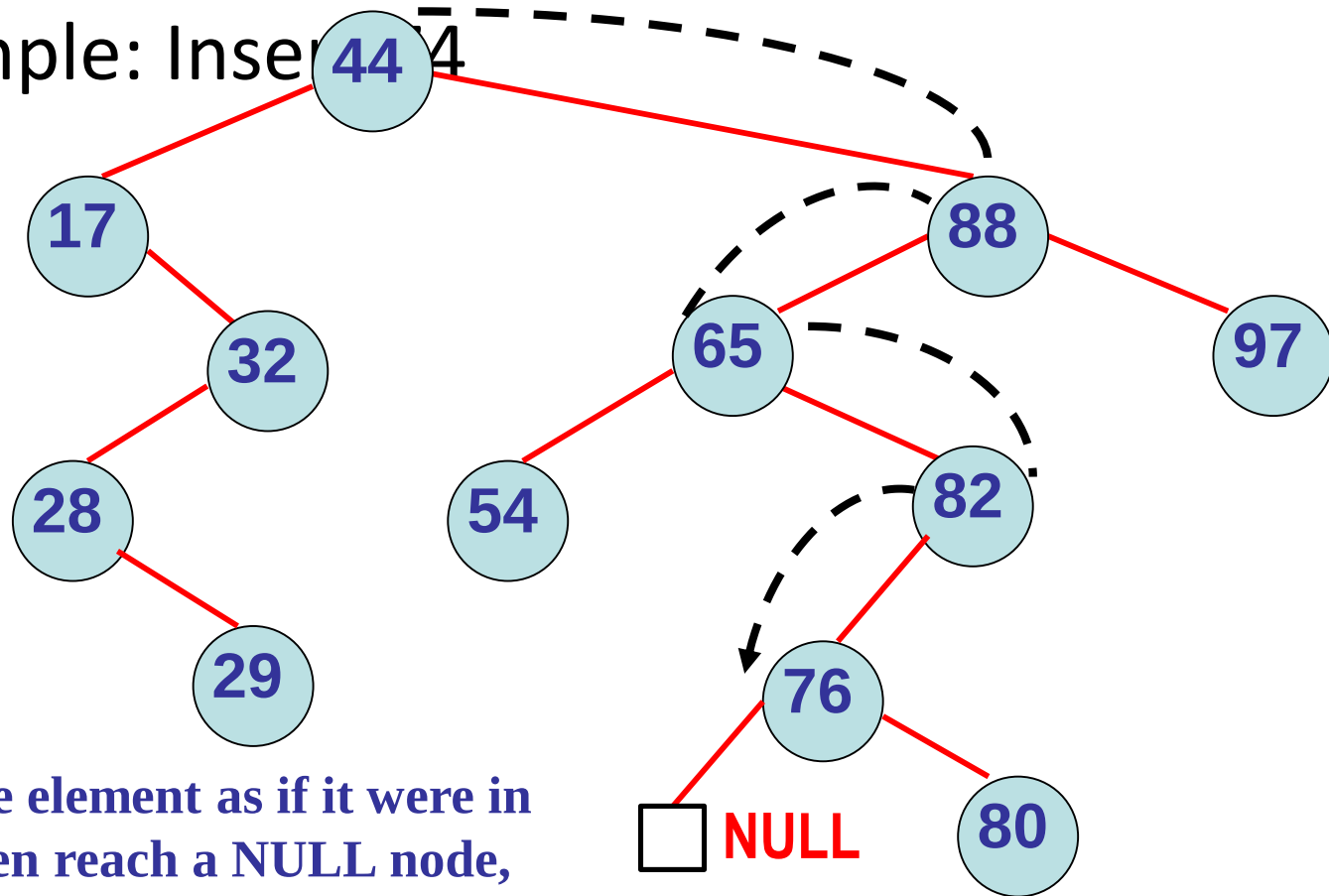
Operations of BSTs: Insert

Insert **35**



Operations of BSTs: Insert

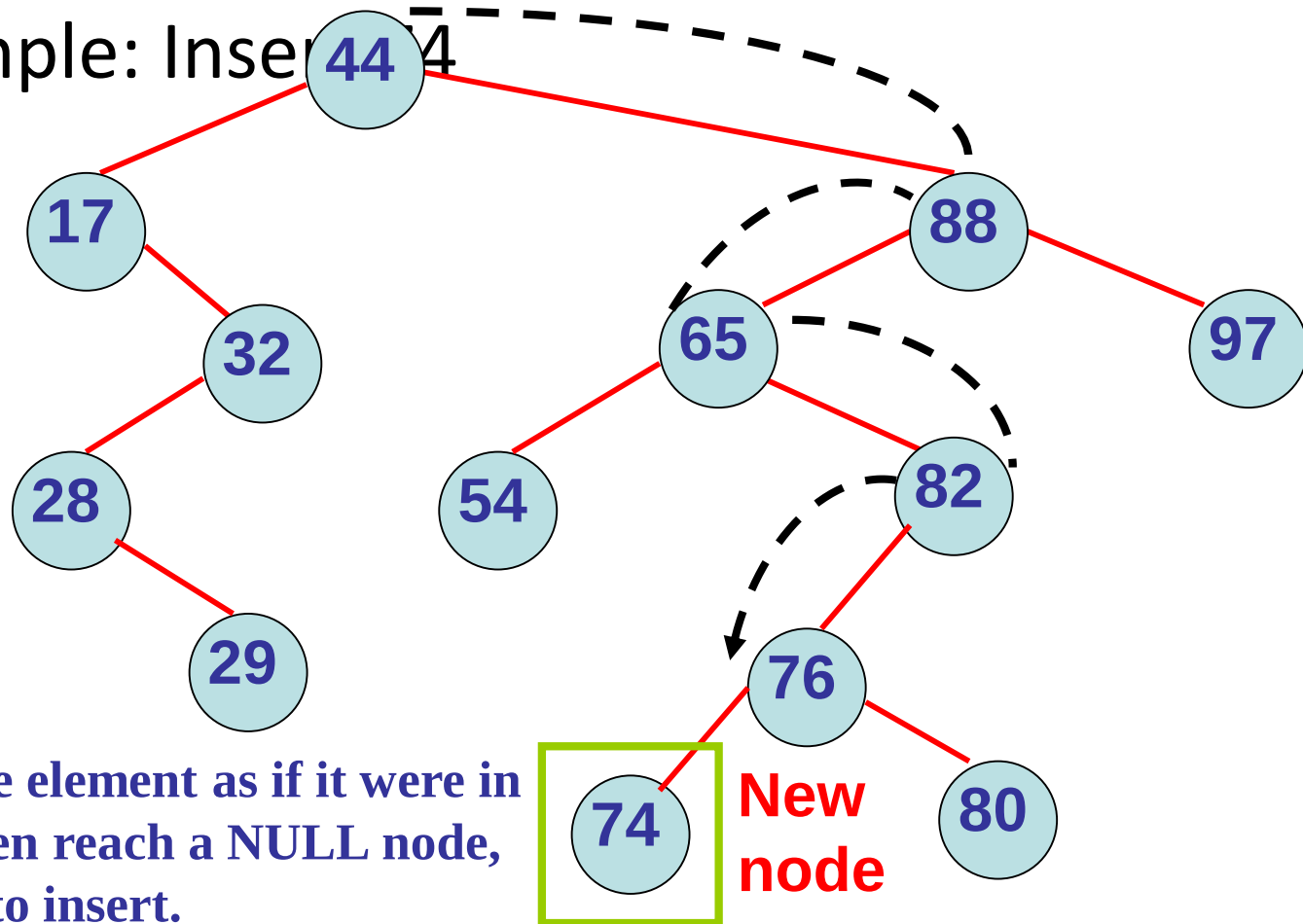
- Example: Insert 44



Search for the element as if it were in the tree. When reach a NULL node, that's where to insert.

Operations of BSTs: Insert

- Example: Insert 74



Operations of BSTs: Insert

TREE-INSERT(T, z)

```
1   $y \leftarrow \text{NIL}$ 
2   $x \leftarrow \text{root}[T]$ 
3  while  $x \neq \text{NIL}$ 
4      do  $y \leftarrow x$ 
5          if  $\text{key}[z] < \text{key}[x]$ 
6              then  $x \leftarrow \text{left}[x]$ 
7              else  $x \leftarrow \text{right}[x]$ 
8   $p[z] \leftarrow y$ 
9  if  $y = \text{NIL}$ 
10     then  $\text{root}[T] \leftarrow z$ 
11     else if  $\text{key}[z] < \text{key}[y]$ 
12         then  $\text{left}[y] \leftarrow z$ 
13         else  $\text{right}[y] \leftarrow z$ 
```

Use Tree-search !

▷ Tree T was empty

Operations of BSTs: Delete

- Deletion is a bit tricky
- 3 cases:

– x has no children:

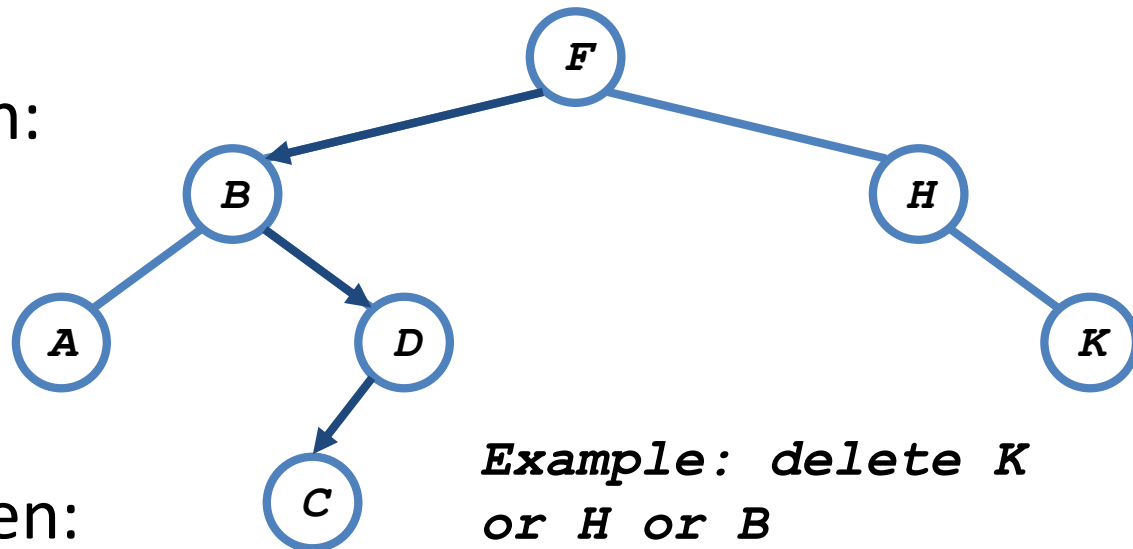
- Remove x

– x has one child:

- Splice out x

– x has two children:

- Find its inorder successor y
- Replace x with y
- Delete y

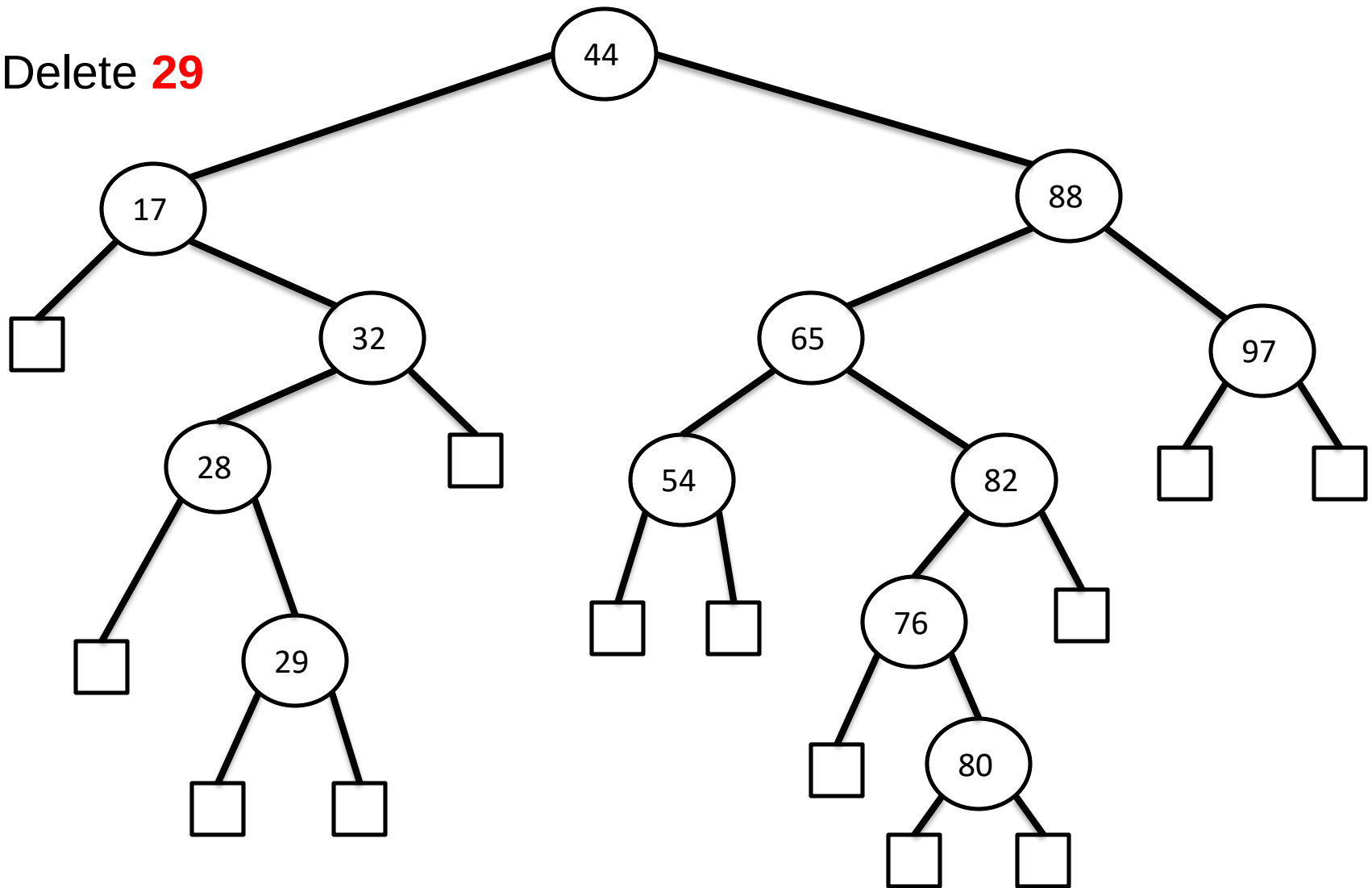


*Example: delete K
or H or B*

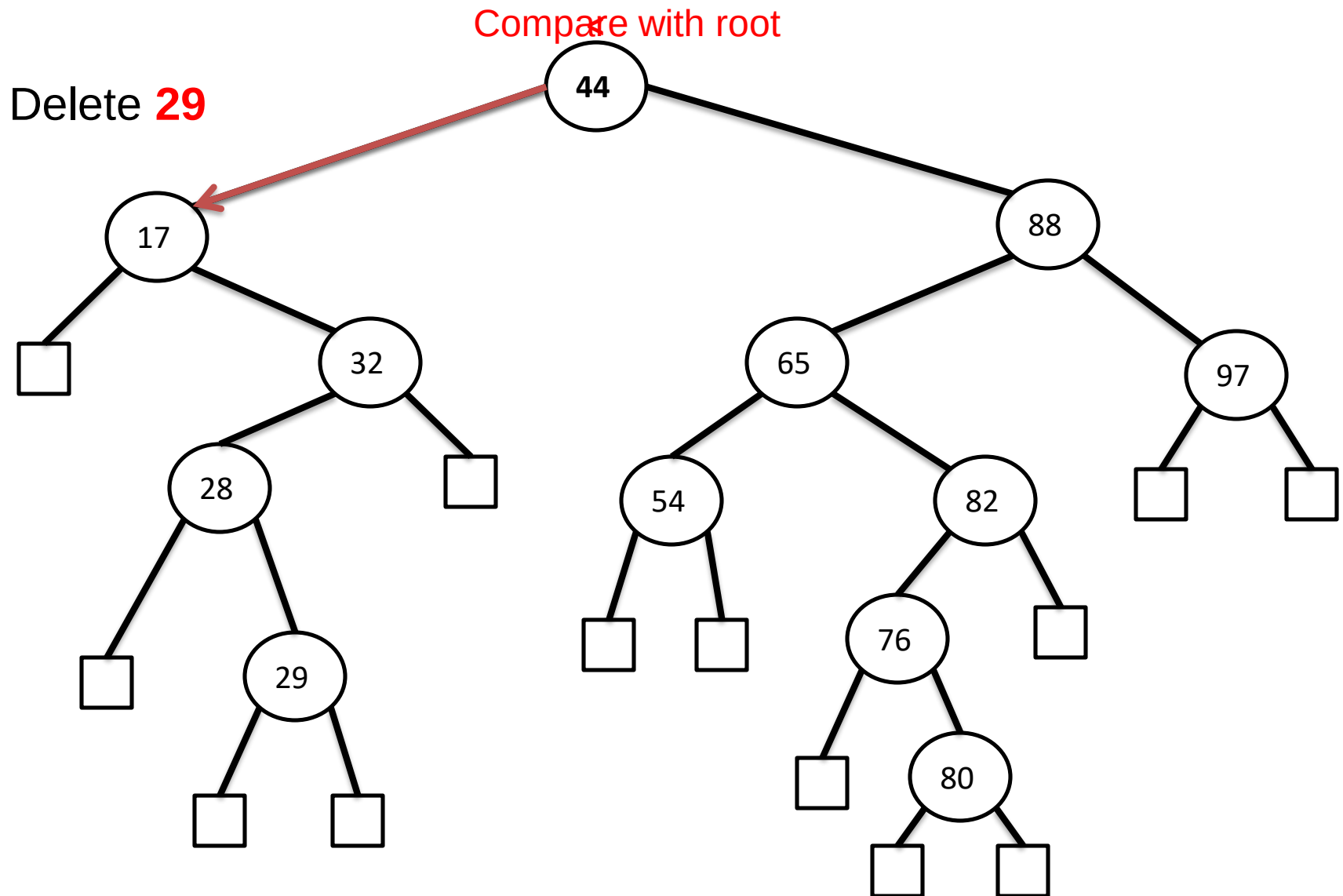
y cannot have left child !

Operations of BSTs: Delete

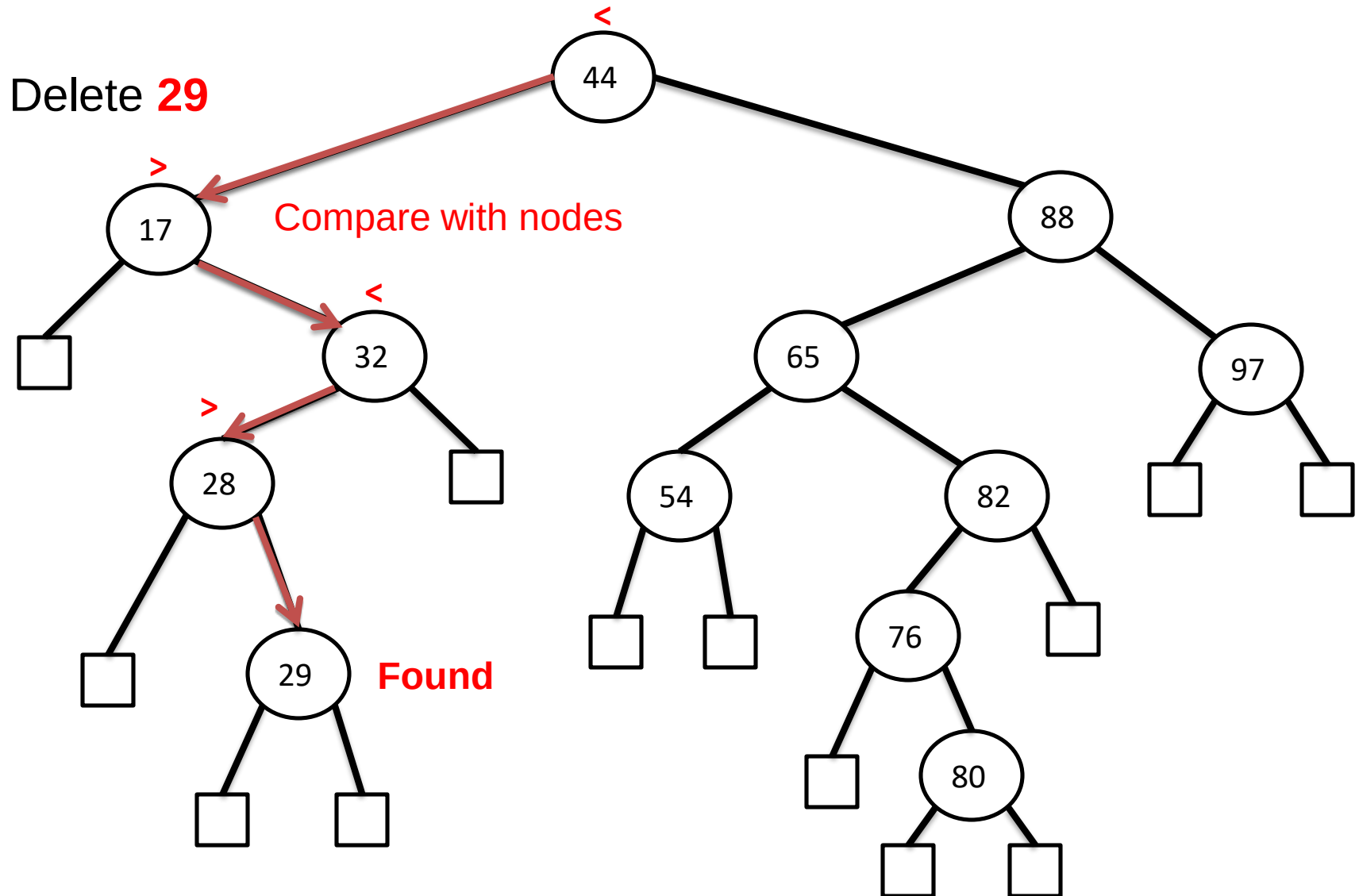
Delete **29**



Operations of BSTs: Delete

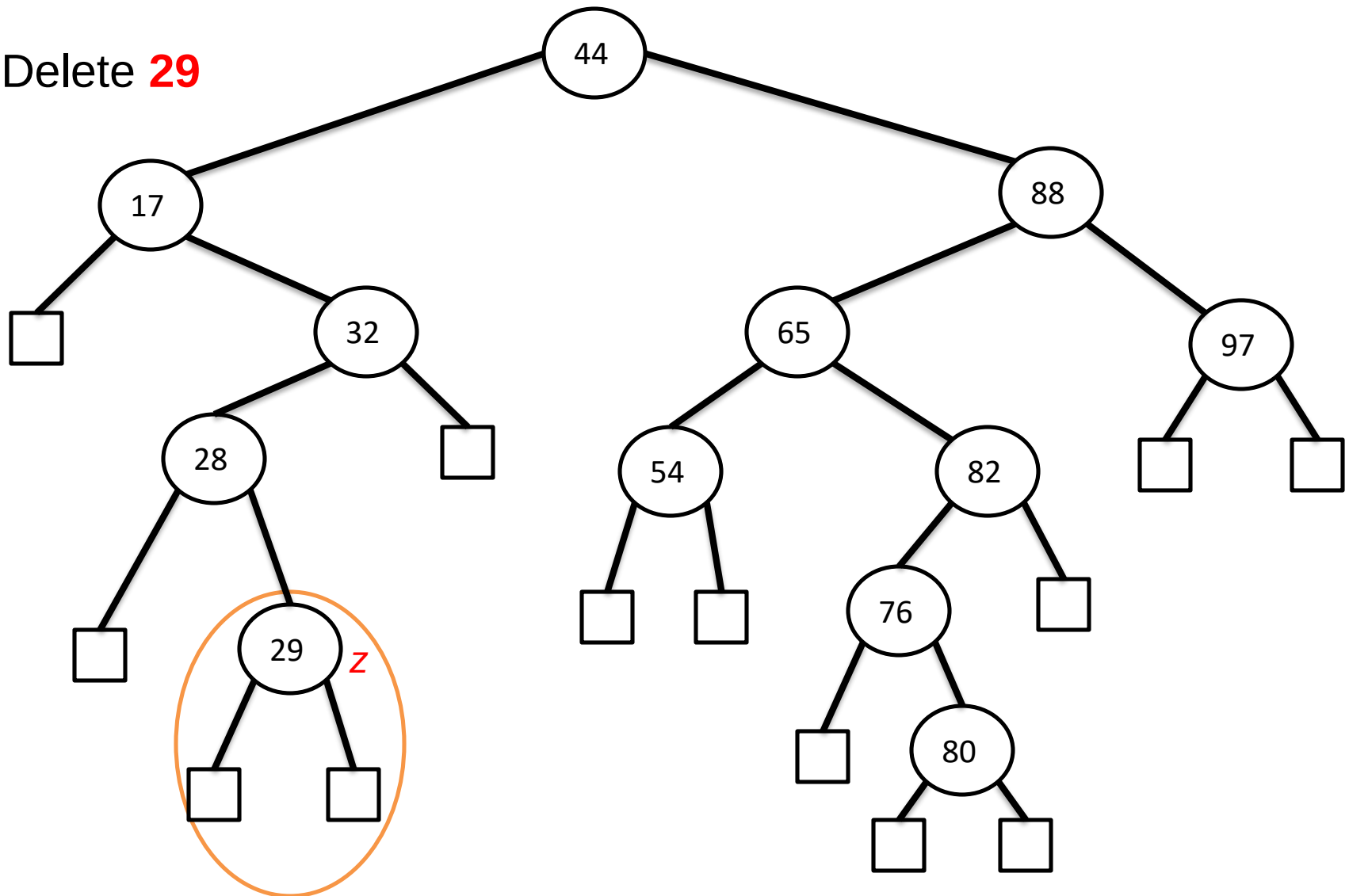


Operations of BSTs: Delete



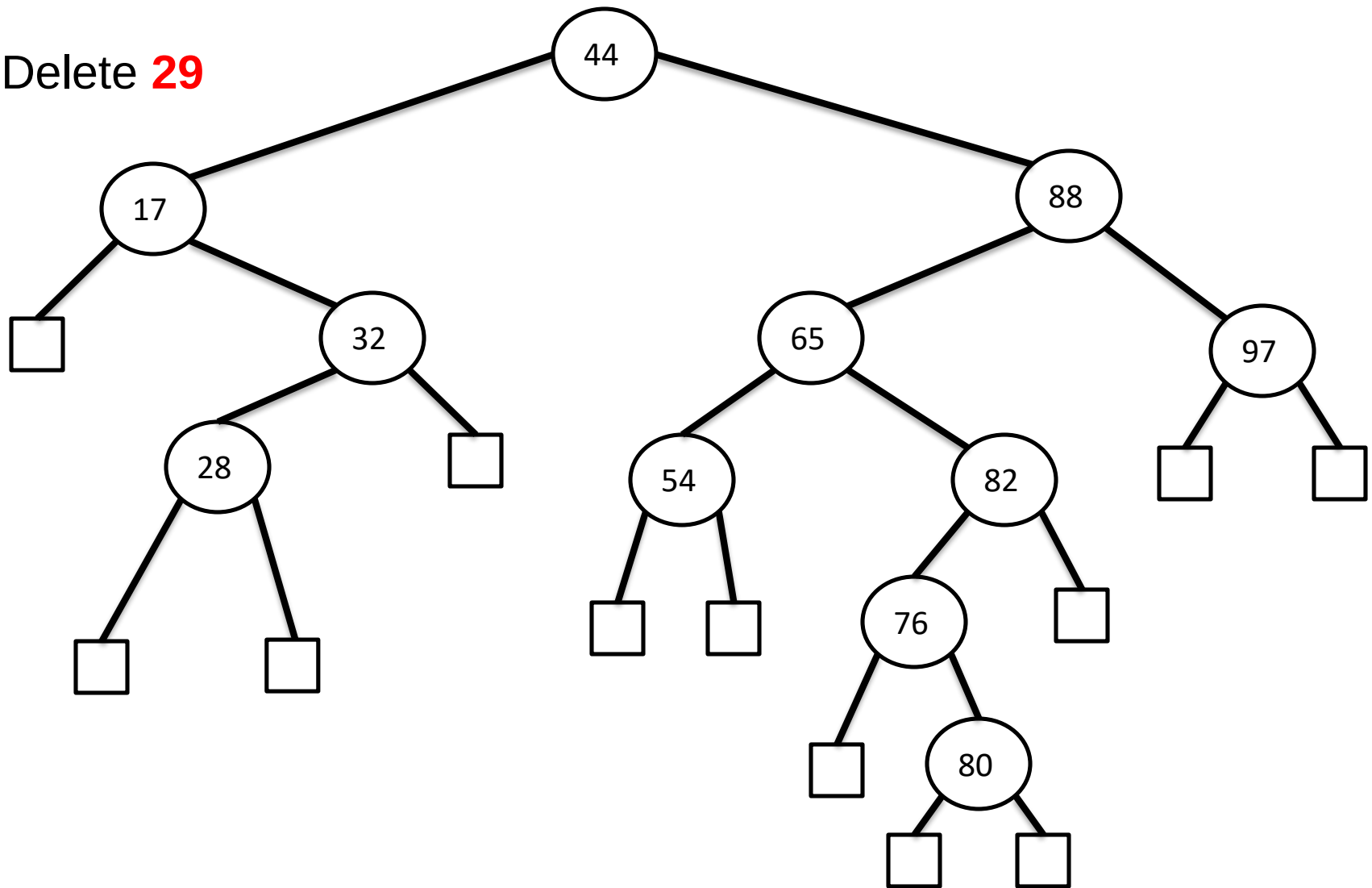
Operations of BSTs: Delete

Delete **29**



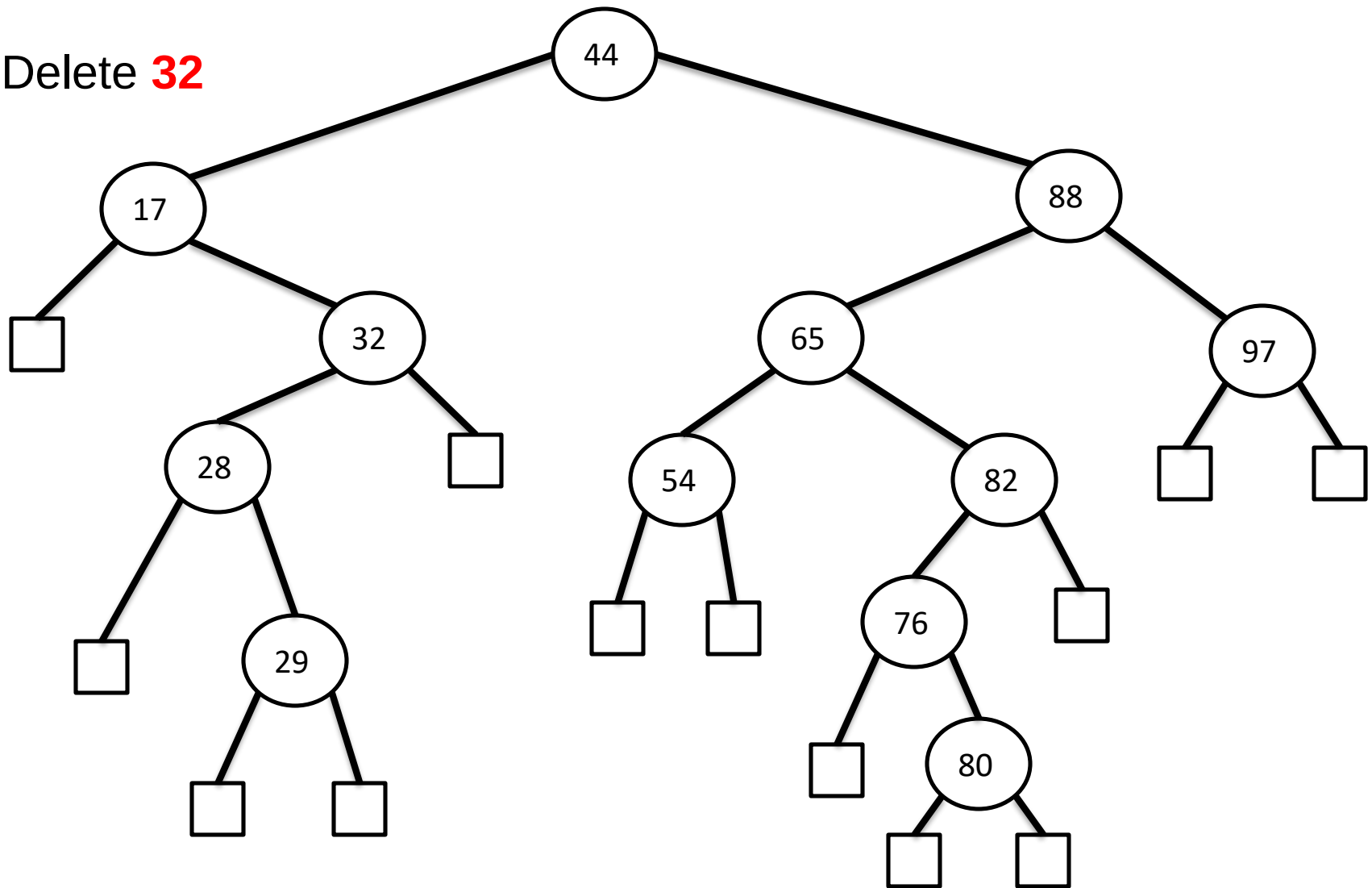
Operations of BSTs: Delete

Delete **29**

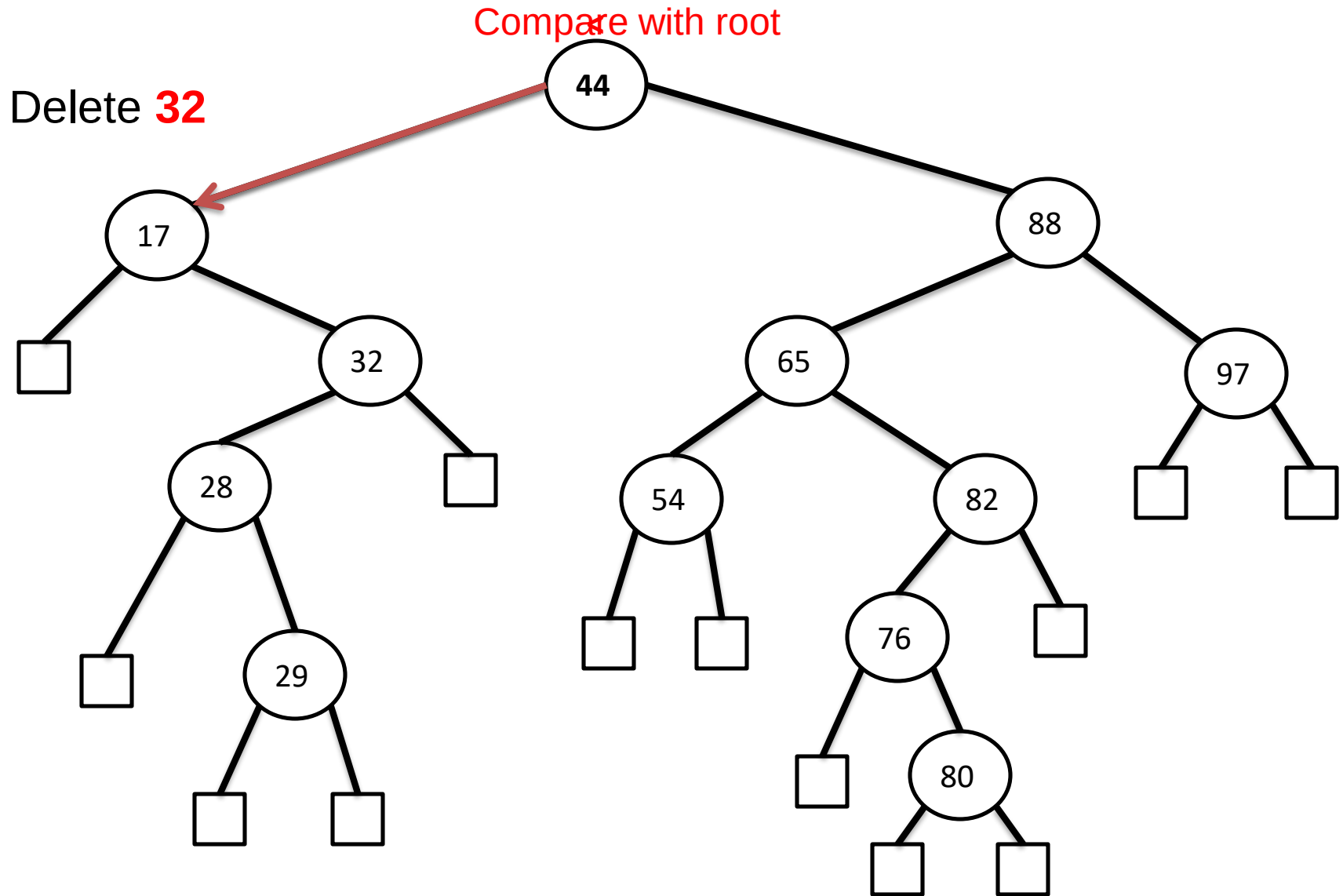


Operations of BSTs: Delete

Delete **32**



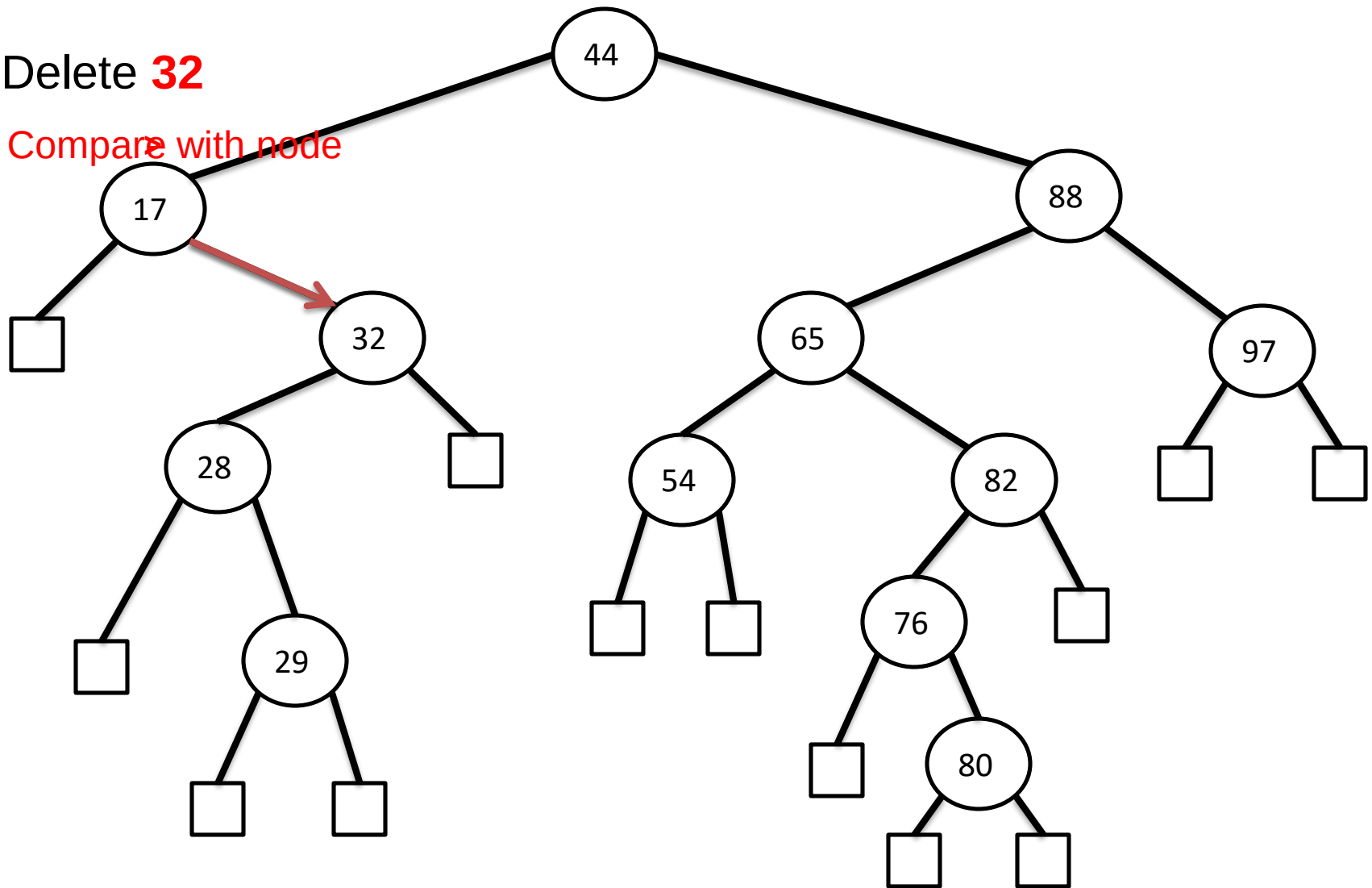
Operations of BSTs: Delete



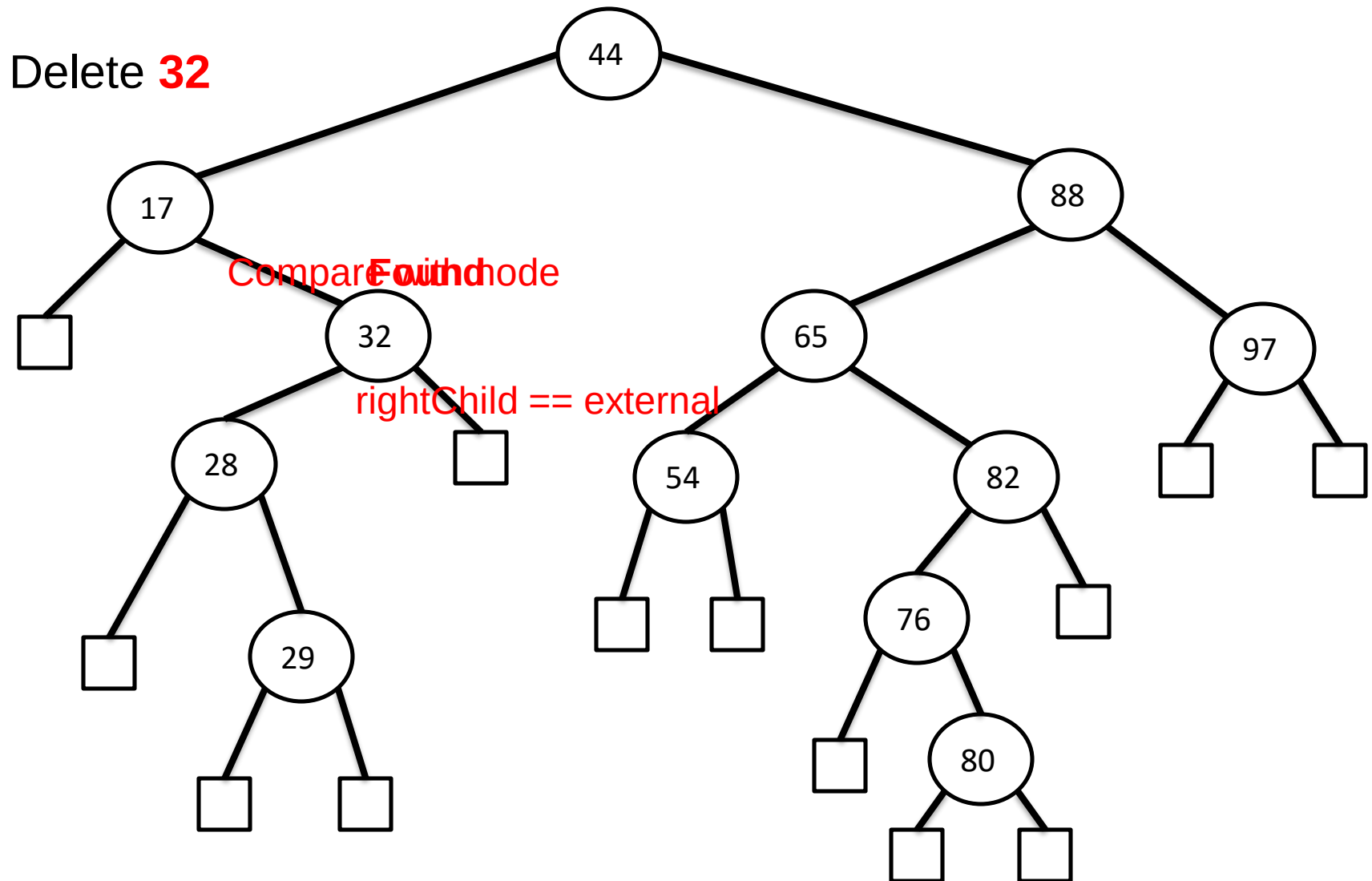
Operations of BSTs: Delete

Delete **32**

Compare with node

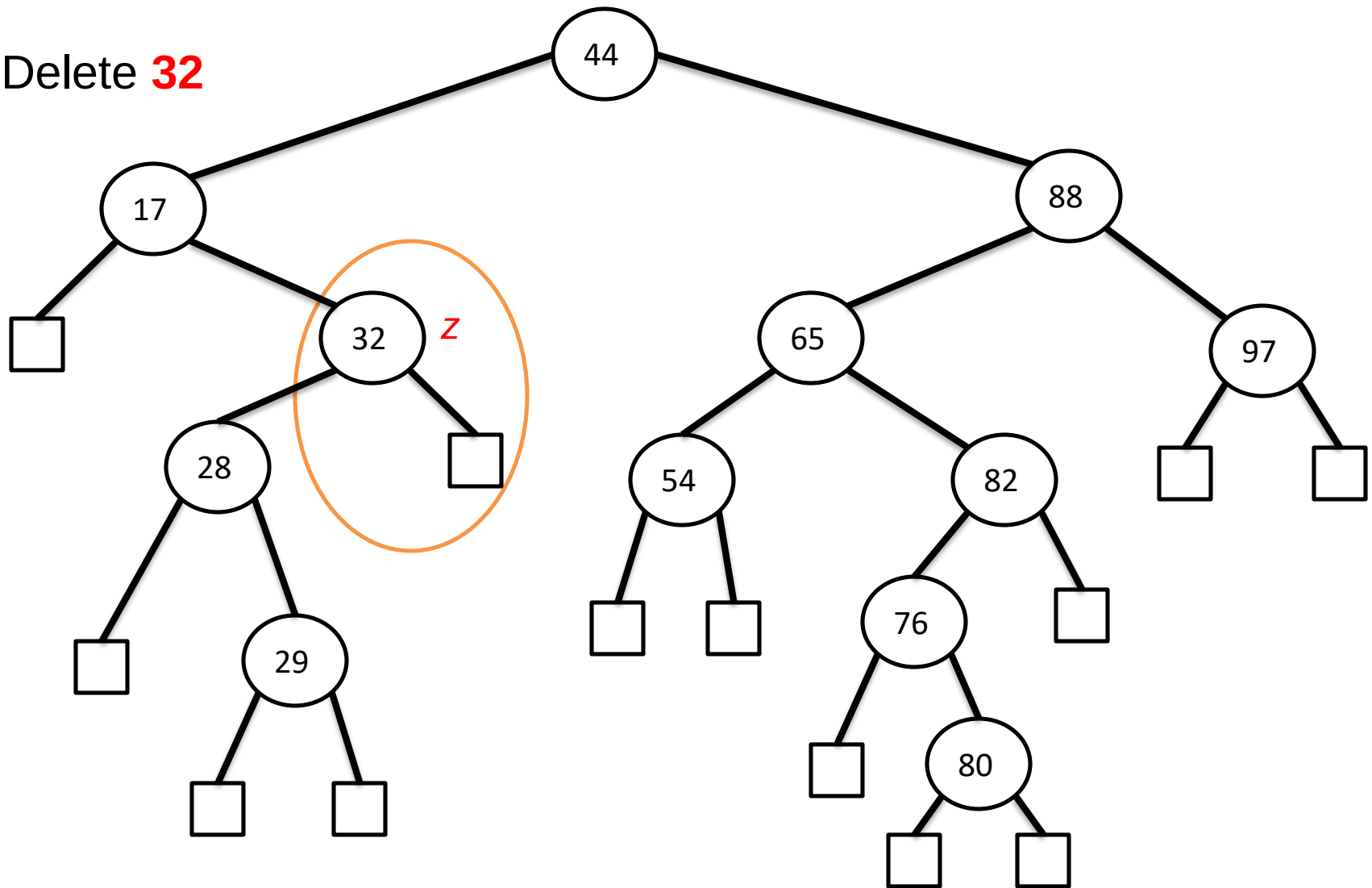


Operations of BSTs: Delete



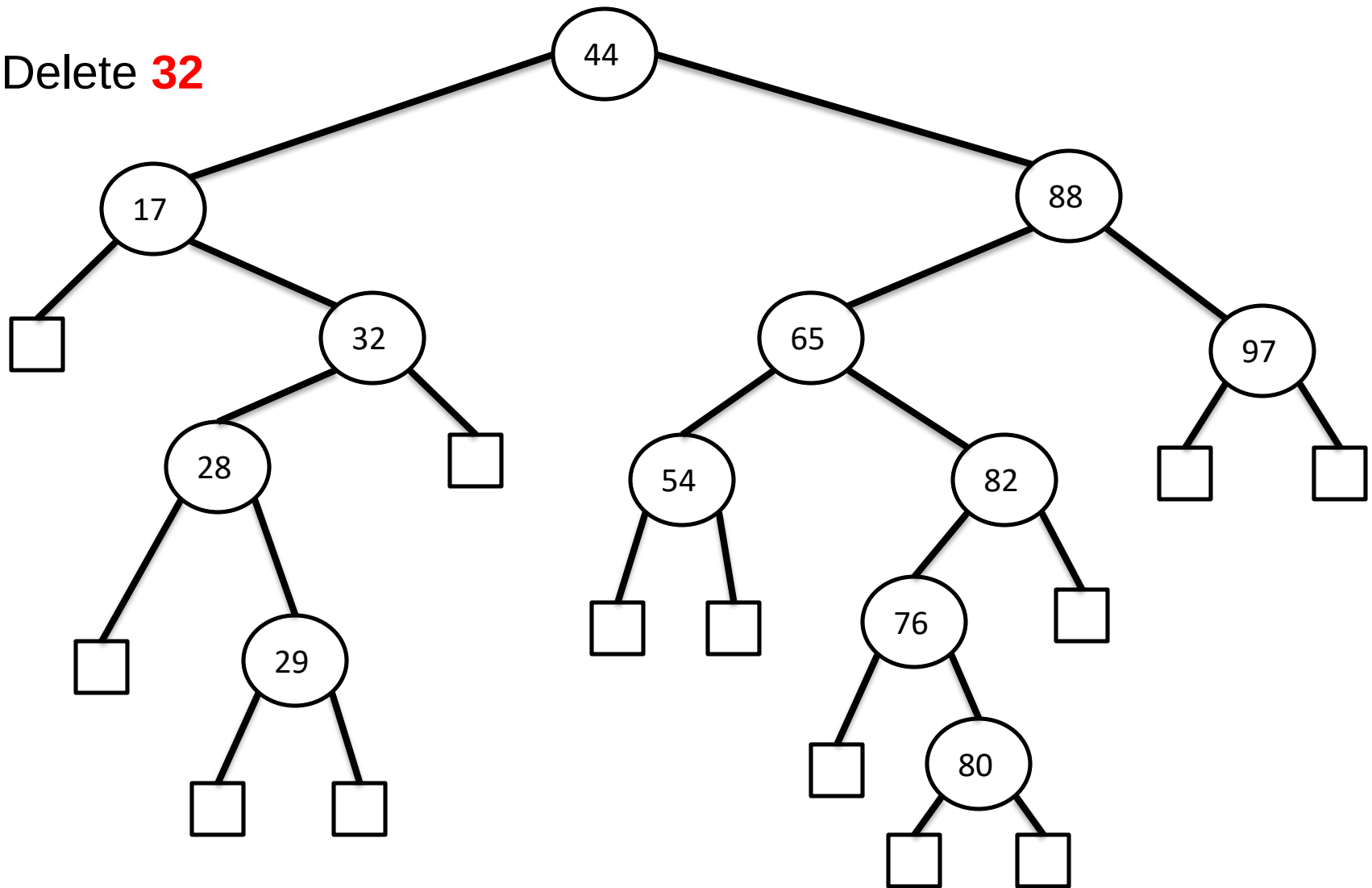
Operations of BSTs: Delete

Delete **32**



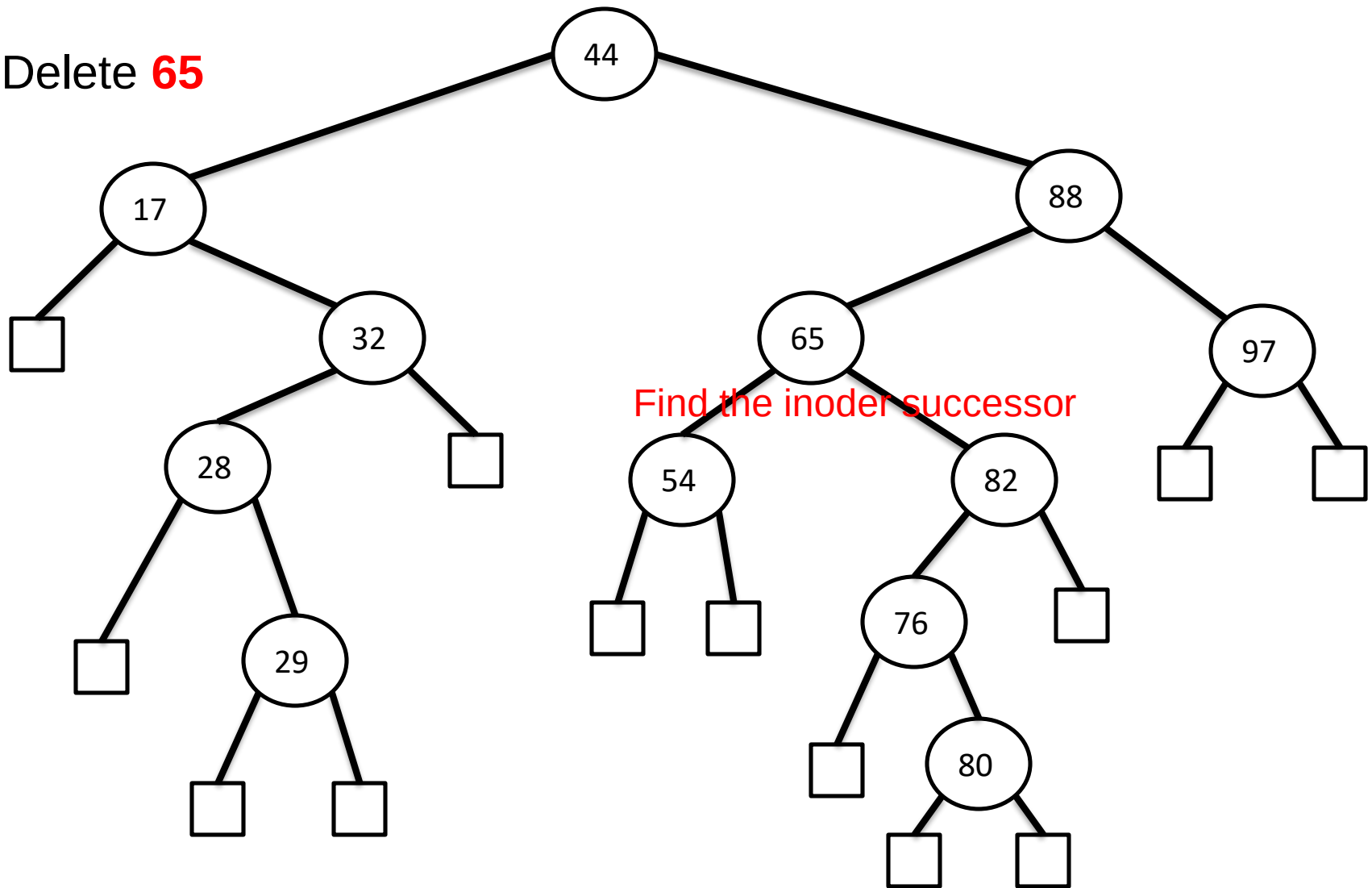
Operations of BSTs: Delete

Delete **32**



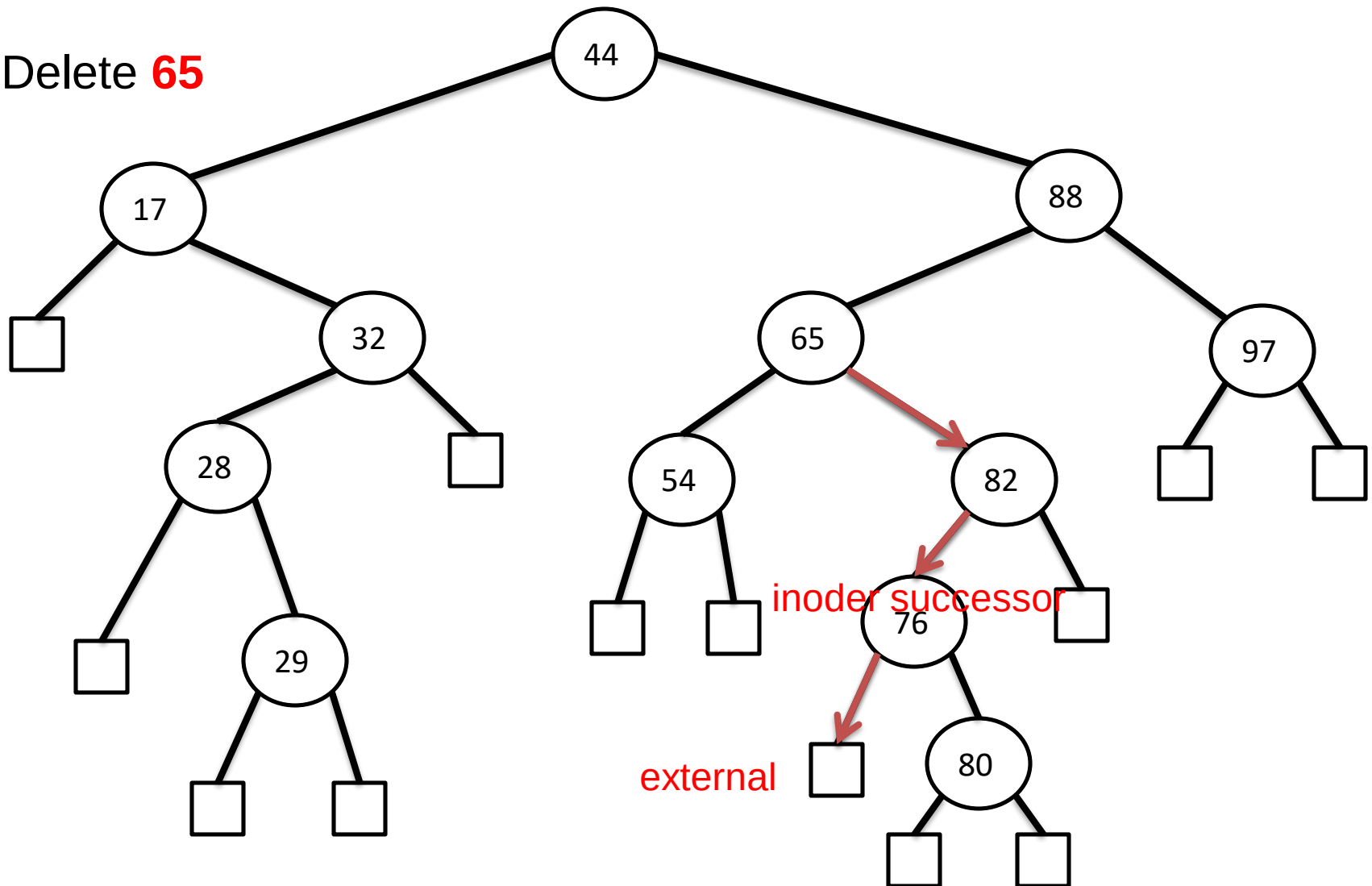
Operations of BSTs: Delete

Delete **65**



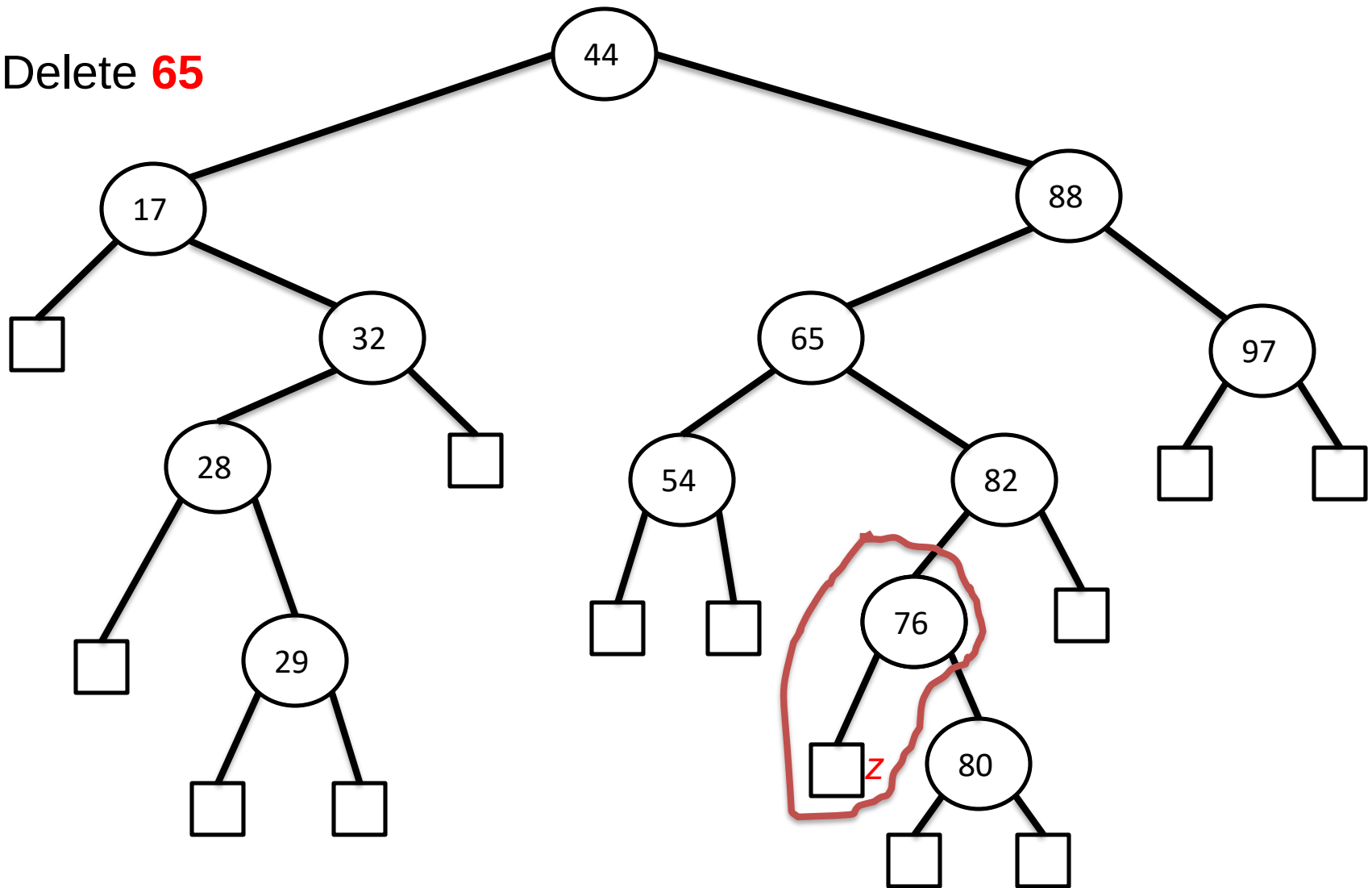
Operations of BSTs: Delete

Delete **65**



Operations of BSTs: Delete

Delete **65**



Operations of BSTs: Delete

- Given two nodes u and v , Transplant replaces the subtree rooted at u with the subtree rooted at v : node u 's parent becomes node v 's parent.

```
Transplant(T, u, v)
    if (p[u] == NIL)
        root[T] = v
    elseif (u == left[ p[u] ])
        left[ p[u] ] = v;
    else right[ p[u] ] = v;
    if v != NIL
        p[v] = p[u];
```

Operations of BSTs:

- Deletes node z from binary search tree T

Tree-Delete(T, z)

if ($\text{left}[z] == \text{NIL}$)

 Transplant($T, z, \text{right}[z]$)

elseif ($\text{right}[z] == \text{NIL}$)

 Transplant($T, z, \text{left}[z]$)

else $y = \text{Tree-Minimum}(\text{right}[z])$

 if $p[y] \neq z$

 Transplant($T, y, \text{right}[y]$)

$\text{right}[y] = \text{right}[z]$

$p[\text{right}[y]] = y$

 Transplant(T, z, y) // $p[y] == z$

$\text{left}[y] = \text{left}[z]$

$p[\text{left}[y]] = y$

Sorting with BSTs

- Informal code for sorting array A of length n :

```
BSTSort (A)
```

```
    for  $i=1$  to  $n$ 
```

```
        TreeInsert ( $A[i]$ ) ;
```

```
    InorderTreeWalk (root) ;
```

- *Running time is $\Omega(n \lg n)$*
- *What will be the running time in the*
 - *Worst case?*
 - *Average case?*

BST: Remarks

- All complexity depends on height h
- $h = \Omega(\lg n)$
- To guarantee performance:
 - Balanced tree !
- Randomly build tree
 - **Theorem**: The expected height of a randomly built binary search tree on n distinct keys is $O(\lg n)$

Balanced Binary Search Trees

- Keep $O(\lg n)$ height under dynamic operations
- General framework:
 - First a binary search tree
 - Maintain properties that ensure height guarantee
- AVL tree, red-black tree, ...